

Table of Contents

1. Background of Project	4
1.1 Introduction.....	4
1.2 Problem Statements	4
1.3 Objectives	5
1.4 Data Preprocessing.....	7
1.4.1 Data Cleaning.....	7
1.4.2 Data Transformation	15
1.4.3 Data Integration	18
1.4.4 Data Reduction.....	19
1.4.5 Preprocessing Summary.....	20
2. Implementation of Data Mining Techniques	22
2.1 Classification: Random Forest vs. Support Vector Machine.....	22
2.1.1 Why Classification?	22
2.1.2 Why Random Forest?	22
2.1.3 Why SVM as Comparison?	23
2.1.4 Data Preparation.....	24
2.1.5 Random Forest with GridSearchCV	25
2.1.6 SVM with GridSearchCV	27
2.1.7 Confusion Matrix Comparison	28
2.2 Prediction: Linear vs. Polynomial Regression.....	30
2.2.1 Why Regression?	30
2.2.2 Why Linear Regression?.....	30
2.2.3 Why Polynomial Regression (Degree 2)?.....	30
2.2.4 Yearly Aggregated Data	31
2.2.5 Linear Regression Results.....	31
2.2.6 Polynomial Regression Results and Comparison	32
2.3 Clustering: K-Means vs. DBSCAN	36
2.3.1 Why Clustering?	36
2.3.2 Why K-Means?	36
2.3.3 Why DBSCAN?.....	37
2.3.4 Data Preparation.....	38
2.3.5 K-Means: Optimal k Selection.....	38

2.3.6 K-Means Results (k=11).....	40
2.3.7 DBSCAN: Two-Phase Approach	41
2.3.8 Visual Comparison.....	43
2.4 Anomaly Detection: Isolation Forest vs. LOF	44
2.4.1 Why Anomaly Detection?.....	44
2.4.2 Why Isolation Forest?	44
2.4.3 Why LOF?	44
2.4.4 Isolation Forest Implementation	45
2.4.5 LOF Implementation.....	46
3. Evaluation and Validation of Models	48
3.1 Classification Model Evaluation.....	48
3.1.1 5-Fold Stratified Cross-Validation.....	48
3.1.2 ROC Curve and AUC	50
3.2 Regression Model Evaluation	51
3.3 Clustering Evaluation.....	54
4. Data Visualisation and Interpretation	59
4.1 Conventional Visualisations	59
4.1.1 Yearly Drug Overdose Death Trends	59
4.1.2 Demographic Breakdown	60
4.1.3 Feature Importance	62
4.1.4 Drug Co-occurrence Heatmap	63
4.1.5 Geographic Cluster Map	65
4.1.6 Anomaly Detection Scatter Plots	66
5. Conclusion	69
5.1 Summary of Findings.....	69
5.2 Limitations	70
5.3 Suggestions for Future Work	71
6. Work Breakdown Structure	73
References.....	74

1. Background of Project

1.1 Introduction

The drug overdose crisis is one of the most serious public health issues in the United States that kills tens of thousands of people each year and places a huge burden on the healthcare, policing, and community. It is important to know the trends of drug-related deaths in order to develop effective and specific interventions. In this report, a detailed data mining analysis of the Accidental Drug-Related Deaths data is conducted on the publicly available data of Connecticut Office of the Chief Medical Examiner, which is one of the public health surveillance datasets containing all accidental drug-related deaths in Connecticut between years 2012 and 2023.

The data has 11,981 entries with 48 attributes including demographic data (age, sex, race, ethnicity), geographic data (residence, injury, and death location with geocoded location), detection of substances (23 drug columns) and description of the manner of death and cause of death. This multi-dimensional, rich data is a rare chance to implement various data mining techniques to generate actionable data that will guide the policy of the public health and resource allocation.

The project implements four fundamental data mining methods, namely classification, regression, clustering and anomaly detection, each of which has a different problem statement. Each technique is applied using two algorithms, which allow performing a strict analysis of the application of those methods that are most appropriate according to the specifics of this dataset.

1.2 Problem Statements

Problem 1: The prediction of the Opioid Involvement in Overdose Deaths.

Is it possible to categorically determine a drug-related death involving opioids according to demographic features and other substance variables? This difference between opioid-involved and non-opioid deaths is essential since opioid-involved deaths should be approached fundamentally differently, such as with the naloxone distribution programmes, medication-assisted treatment (MAT) facilities, and specific prescriber education. When we formulate it as a binary classification task, we hope to find out the most important predictors that can differentiate opioid and non-opioid

deaths. The method chosen is classification, the comparison between the Random Forest and Support Vector Machine (SVM).

Problem 2: Drug Overdose Substance Trends Modelling.

What are the patterns of the changes in substance involvement with time, and are they modifiable? The drug market has not been stagnant, as since 2014, the level of fentanyl has skyrocketed, heroin has become rise-then-fall, cocaine engagement has continued to rise, and xylazine has become a threatening new adulterant since 2019. The knowledge of these time dynamics allows resource pre-allocation. This is outlined as a regression exercise, in a comparison of Linear Regression and Polynomial Regression (degree 2).

Problem 3: Tracking Geographic Clusters of Hotspots of Overdose.

Are deaths associated with drugs geographically clustered in particular areas and can we detect particular clusters of hotspots? The location of naloxone distribution centres, the mobile treatment facilities, and community outreach programmes can be directed directly through the analysis through spatial clustering. This is developed as an unsupervised clustering issue based on geographic coordinates, which compares K-Means and DBSCAN.

Problem 4: The identification of Anomalous cases of overdose.

Are there exceptional cases which are not in the majority pattern? Abnormal cases can indicate new drug threats or distinctive combinations of substances or demographic changes which should be intercepted at an earlier stage. This is posed as an anomaly detection challenge in terms of 10 features between Isolation Forest and Local Outlier Factor (LOF).

1.3 Objectives

Table 1 provides the correspondence of each problem statement with its objective and data mining methods used.

#	Problem	Objective	Algorithm 1	Algorithm 2
1	Predicting opioid involvement	Classify opioid vs non-opioid deaths	Random Forest	SVM

2	Substance trend modelling	Predict yearly substance counts	Linear Regression	Polynomial Regression
3	Geographic hotspot identification	Discover spatial death clusters	K-Means	DBSCAN
4	Anomalous case detection	Identify rare unusual overdose cases	Isolation Forest	LOF

Table 1: Problem–Objective–Technique Alignment

1.4 Data Preprocessing

Before the raw data could be utilized in the data mining, it underwent a lot of preprocessing. Four types of preprocessing were used: (1) Data Cleaning- fix inconsistencies, standardise values, and handle missing data; (2) Data Transformation- feature engineering, encoding and scaling; (3) Data Integration- extract geographic coordinates out of geocoded strings and (4) Data Reduction- remove redundant and irrelevant columns. All the steps will be recorded below with the code screenshots illustrating how they are implemented and the outputs prior to and after the implementation.

1.4.1 Data Cleaning

Step 3: Resolve Column Name White Space. The column Other Significant Conditions had a trailing whitespace character and so when used as a key, it raised KeyError exceptions. The pandas `str.strip` was used to strip all column **names**.

```
In [6]:
1 # Step 3: Fix column name whitespace
2 # Issue: Some column names have trailing spaces (e.g., 'Other Significant Conditions ')
3 # which causes KeyError when referencing columns
4 print("BEFORE - Columns with whitespace issues:")
5 whitespace_issues = [col for col in df.columns if col != col.strip()]
6 for col in whitespace_issues:
7     print(f" Found: '{col}' (trailing/leading whitespace)")
8 if not whitespace_issues:
9     print(" None found")
10
11 df.columns = df.columns.str.strip()
12
13 print(f"\nAFTER - All column names stripped. Total columns: {len(df.columns)}")
14
```

Output:

```
BEFORE - Columns with whitespace issues:
 Found: 'Other Significant Conditions ' (trailing/leading whitespace)

AFTER - All column names stripped. Total columns: 48
```

Code Snippet 1: Stripping trailing whitespace from column names to prevent KeyError exceptions (Step 3).

Step 4: Standardise Manner of Death. Inconsistent casing ('Accident', 'accident', 'ACCIDENT') and a typographical error ('Accident') were standardised to Title Case.

```

In [7]:
1 # Step 4: Standardise 'Manner of Death' column
2 # Issue: Contains inconsistent casing ('Accident', 'accident', 'ACCIDENT') and typo ('Accident')
3 print("BEFORE – Manner of Death value counts:")
4 print(df['Manner of Death'].value_counts().to_string())
5
6 df['Manner of Death'] = df['Manner of Death'].str.strip().str.title()
7
8 print("\nAFTER – Manner of Death value counts (standardised to Title Case):")
9 print(df['Manner of Death'].value_counts().to_string())
10

```

Output:

```

BEFORE – Manner of Death value counts:
Manner of Death
Accident      11942
Pending        14
accident       13
ACCIDENT        1
Natural         1
Accident        1

AFTER – Manner of Death value counts (standardised to Title Case):
Manner of Death
Accident      11956
Pending        14
Natural         1
Accident        1

```

Code Snippet 2: Standardising inconsistent casing and typographical errors in the Manner of Death column (Step 4).

Step 5: Standardise Race. Over 15 fragmented categories including typographical errors were consolidated into 6 major categories for statistical validity.

In [8]:

```
1 # Step 5: Standardise 'Race' column
2 # Issue: Too many fragmented categories (15+) including typos and rare sub-groups
3 # Strategy: Aggressively consolidate into 5 major categories for statistical validity
4 # This ensures LabelEncoder produces meaningful ordinal values
5 print("BEFORE – Race value counts:")
6 print(df['Race'].value_counts().to_string())
7
8 df['Race'] = df['Race'].str.strip()
9
10 # Step 5a: Fix case inconsistencies first
11 df['Race'] = df['Race'].str.title() # Normalise casing
12
13 # Step 5b: Map all variants to 5 major categories
14 race_mapping = {
15     'Black': 'Black or African American',
16     'Black Or African American': 'Black or African American',
17     'Black Or African American / American Indian Lenni Lenape': 'Black or African American',
18     'Asian Indian': 'Asian',
19     'Chinese': 'Asian',
20     'Korean': 'Asian',
21     'Asian, Other': 'Asian',
22     'Asian/Indian': 'Asian',
23     'Other Asian': 'Asian',
24     'Other Asian (Specify)': 'Asian',
25     'Hawaiian': 'Native Hawaiian or Other Pacific Islander',
26     'Native Hawaiian Or Other Pacific Islander': 'Native Hawaiian or Other Pacific Islander',
27     'Native American, Other': 'American Indian or Alaska Native',
28     'American Indian Or Alaska Native': 'American Indian or Alaska Native',
29     'Other': 'Other/Unknown',
30     'Unknown': 'Other/Unknown',
31     'Other (Specify)': 'Other/Unknown',
32     'Other (Specify) Haitian': 'Other/Unknown',
33     'Other (Specify) Puerto Rican': 'Other/Unknown',
34     'Other (Specify) Portugese, Cape Verdean': 'Other/Unknown',
35     'Other/Unknown': 'Other/Unknown'
36 }
37 df['Race'] = df['Race'].replace(race_mapping)
38
39 # Catch any remaining rare categories
40 major_races = ['White', 'Black or African American', 'Asian',
41                'American Indian or Alaska Native',
42                'Native Hawaiian or Other Pacific Islander', 'Other/Unknown']
43 df.loc[~df['Race'].isin(major_races), 'Race'] = 'Other/Unknown'
44
45 print("\nAFTER – Race value counts (consolidated to 6 categories):")
46 # ... (continued)
```

Output:

```
BEFORE – Race value counts:
Race
White                                10080
Black or African American             826
Black                                 809
Unknown                               62
Other                                 52
Asian Indian                          26
Asian, Other                          24
Other (Specify)                       13
Other Asian                           12
Asian/Indian                           5
Chinese                                2
white                                  2
Asian                                  2
American Indian or Alaska Native       1
Hawaiian                              1
Native American, Other                 1
Korean                                1
Other Asian (Specify)                  1
Other (Specify) Haitian                1
Other (Specify) portugese, Cape Verdean 1
Other (Specify) Puerto Rican           1
Black or African American / American Indian Lenni Lenape 1

AFTER – Race value counts (consolidated to 6 categories):
Race
White                                10082
Black or African American            1636
Other/Unknown                         187
Asian                                 73
American Indian or Alaska Native       2
Native Hawaiian or Other Pacific Islander 1

Unique categories: 6
```

Code Snippet 3: Consolidating 15+ fragmented race categories into 6 standard classifications (Step 5).

Step 6: Standardise Sex. ‘Unknown’ (2) and ‘X’ (1) entries were replaced with NaN. After cleaning: Male: 8,887 (74.2%), Female: 3,082 (25.8%).

```
In [9]:
1 # Step 6: Standardise 'Sex' column
2 # Issue: 'Unknown' (2 entries) and 'X' (1 entry) cannot be used for analysis
3 # Strategy: Replace with NaN – these 3 records will be dropped in Step 11
4 print("BEFORE – Sex value counts:")
5 print(df['Sex'].value_counts().to_string())
6
7 df['Sex'] = df['Sex'].replace({'Unknown': np.nan, 'X': np.nan})
8
9 print("\nAFTER – Sex value counts:")
10 print(df['Sex'].value_counts().to_string())
11 print(f"  NaN count: {df['Sex'].isna().sum()}")
12
```

Output:

```
BEFORE – Sex value counts:
Sex
Male      8887
Female    3082
Unknown     2
X           1

AFTER – Sex value counts:
Sex
Male      8887
Female    3082
NaN count: 12
```

Code Snippet 4: Replacing invalid Sex values (Unknown, X) with NaN and validating final distribution (Step 6).

Step 7: Standardise Ethnicity. 13 different string values were mapped to 3 categories: Hispanic (1,740), Non-Hispanic (788), Unknown (9,453).

The field of ethnicity was one of the major standardisation issues: the raw data had 13 different items of the form of a string with different variations, like Hispanic or Latino; Hispanic/Latino; Non Hispanic; Non-Hispanic; and several capitalisation forms. The fact that each is treated as a separate category would have disaggregated an already small variable into insignificant categories. This is because the three-values consolidation (Hispanic, Non-Hispanic, Unknown) is the standard of the U.S. Office of Management and Budget ethnicity classification utilized by federal health agencies, and therefore it is compatible with national standards. The very high percentage of Unknown values (9,453 cases, or 78.9%) does not represent an artifact of preprocessing but is due to the actual gap in the original medical examiner data. This unspecified rate was reported as a

limitation instead of having been inferred, since it would be an unacceptable inferential jump to assign ethnicity on the basis of other demographic variables.

```
In [10]:

1 # Step 7: Standardise 'Ethnicity' column
2 # Issue: 13 different values representing essentially 3 categories
3 # Strategy: Map all variants to Hispanic / Non-Hispanic / Unknown
4 print("BEFORE – Ethnicity value counts:")
5 print(df['Ethnicity'].value_counts().to_string())
6
7 hispanic_values = ['Hispanic', 'Spanish/Hispanic/Latino',
8                   'Other Spanish/Hispanic/Latino',
9                   'Yes, other Spanish/Hispanic/Latino',
10                  'Yes, Other Spanish/Hispanic/Latino (Specify)',
11                  'Puerto Rican', 'Mexican', 'Cuban']
12 non_hispanic_values = ['Not Spanish/Hispanic/Latino',
13                       'No, not Spanish/Hispanic/Latino']
14
15 df['Ethnicity'] = df['Ethnicity'].apply(
16     lambda x: 'Hispanic' if x in hispanic_values
17     else ('Non-Hispanic' if x in non_hispanic_values
18         else 'Unknown'))
19
20 print("\nAFTER – Ethnicity value counts (3 categories):")
21 print(df['Ethnicity'].value_counts().to_string())
22
```

Output:

```
BEFORE – Ethnicity value counts:
Ethnicity
Hispanic                      972
No, not Spanish/Hispanic/Latino 732
Spanish/Hispanic/Latino       267
Other Spanish/Hispanic/Latino  254
Yes, other Spanish/Hispanic/Latino 200
Not Spanish/Hispanic/Latino    56
Yes, Other Spanish/Hispanic/Latino (Specify) 26
Puerto Rican                 19
Unknown                      17
Yes, Puerto Rican             10
Yes, Mexican, Mexican American, Chicano 6
Mexican, Mexican American, Chicano 4
Cuban                        2

AFTER – Ethnicity value counts (3 categories):
Ethnicity
Unknown      9453
Hispanic     1740
Non-Hispanic  788
```

Code Snippet 5: Mapping 13 ethnicity string variants to 3 OMB-aligned categories (Step 7).

Step 8: Standardise Location. Fixed typographical error ‘Hiospital’, merged hospital subcategories into ‘Hospital’, merged Nursing Home into Convalescent Home.

```

In [11]:
1 # Step 8: Standardise 'Location' column
2 # Issue: Typos (e.g., 'Hiospital') and overlapping categories
3 # (e.g., 'Hospital - Inpatient', 'Hospital - ER/Outpatient' can be merged into 'Hospital')
4 print("BEFORE - Location value counts:")
5 print(df['Location'].value_counts().to_string())
6
7 df['Location'] = df['Location'].str.strip()
8 df['Location'] = df['Location'].replace({
9     "Decedent's Home": 'Residence',
10    "Decedent\'s Home": 'Residence',
11    'Hiospital': 'Hospital',
12    'Hospital - Inpatient': 'Hospital',
13    'Hospital - ER/Outpatient': 'Hospital',
14    'Hospital - Dead On Arrival': 'Hospital',
15    'Nursing Home': 'Convalescent Home',
16    'Other (Specify)': 'Other'
17 })
18
19 print("\nAFTER - Location value counts (consolidated):")
20 print(df['Location'].value_counts().to_string())
21

```

Output:

```

BEFORE - Location value counts:
Location
Residence                4799
Hospital                2336
Other                   1559
Decedent's Home          776
Hospital - ER/Outpatient   401
Other (Specify)          362
Hospital - Inpatient      246
Hospital - Dead On Arrival  81
Decedent's Home           52
Nursing Home              9
Convalescent Home         3
Hospice                   3
Assisted Living           2
Shelter                   1
Hiospital                 1
Hospice Facility          1

AFTER - Location value counts (consolidated):
Location
Residence                4851
Hospital                3065
Other                   1921
Decedent's Home          776
Convalescent Home        12
Hospice                   3
Assisted Living           2
Shelter                   1
Hospice Facility          1

```

Code Snippet 6: Fixing the 'Hiospital' typographical error and merging hospital sub-categories (Step 8).

Step 9: Convert Drug Columns to Binary. 23 drug indicator columns using 'Y'/NaN encoding were converted to 1 (detected) and 0 (not detected) for ML compatibility.

In [12]:

```
1 # Step 9: Convert drug indicator columns to binary (1/0)
2 # Issue: Drug columns use 'Y'/NaN encoding - ML algorithms need numeric values
3 # Strategy: Any non-null value -> 1 (drug detected), null -> 0 (drug not detected)
4 drug_columns = [
5     'Heroin', 'Heroin death certificate (DC)', 'Cocaine', 'Fentanyl',
6     'Fentanyl Analogue', 'Oxycodone', 'Oxymorphone', 'Ethanol', 'Hydrocodone',
7     'Benzodiazepine', 'Methadone', 'Meth/Amphetamine', 'Amphet', 'Tramad',
8     'Hydromorphone', 'Morphine (Not Heroin)', 'Xylazine', 'Gabapentin',
9     'Opiate NOS', 'Heroin/Morph/Codeine', 'Other Opioid', 'Any Opioid', 'Other'
10 ]
11
12 print("BEFORE - Sample drug column values:")
13 for col in ['Fentanyl', 'Ethanol', 'Any Opioid']:
14     print(f" {col}: {df[col].unique()[:5]}")
15
16 for col in drug_columns:
17     if col in df.columns:
18         df[col] = df[col].notna().astype(int)
19
20 print("\nAFTER - Drug columns converted to binary (0/1):")
21 for col in ['Fentanyl', 'Ethanol', 'Any Opioid']:
22     print(f" {col}: unique={df[col].unique()}, positive={df[col].sum():,}")
23
```

Output:

```
BEFORE - Sample drug column values:
  Fentanyl: <StringArray>
[nan, 'Y', 'Y POPS', 'Y (PTCH)']
Length: 4, dtype: str
  Ethanol: <StringArray>
[nan, 'Y', 'P']
Length: 3, dtype: str
  Any Opioid: <StringArray>
[nan, 'Y', 'N']
Length: 3, dtype: str

AFTER - Drug columns converted to binary (0/1):
  Fentanyl: unique=[0 1], positive=8,049
  Ethanol: unique=[0 1], positive=3,201
  Any Opioid: unique=[0 1], positive=8,947
```

Code Snippet 7: Converting 23 drug indicator columns from Y/NaN encoding to binary 1/0 for ML compatibility (Step 9).

Step 10: Date Parsing and Temporal Feature Extraction. Date strings were converted to datetime64 format. Year (2012–2023), Month, and DayOfWeek features were extracted.

```
In [13]:

1 # Step 10: Parse Date column and extract temporal features
2 # Issue: Date stored as string – need datetime for temporal analysis
3 # Strategy: Convert to datetime, then extract Year, Month, DayOfWeek features
4 print(f"BEFORE – Date dtype: {df['Date'].dtype}")
5 print(f" Sample: {df['Date'].head(3).tolist()}")
6
7 df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
8
9 df['Year'] = df['Date'].dt.year
10 df['Month'] = df['Date'].dt.month
11 df['DayOfWeek'] = df['Date'].dt.dayofweek
12
13 print(f"\nAFTER – Date dtype: {df['Date'].dtype}")
14 print(f" Year range: {df['Year'].min():.0f} – {df['Year'].max():.0f}")
15 print(f" Records with valid dates: {df['Date'].notna().sum():,}")
16

Output:

BEFORE – Date dtype: str
Sample: ['05/29/2012', '06/27/2012', '03/24/2014']

AFTER – Date dtype: datetime64[us]
Year range: 2012 – 2023
Records with valid dates: 11,981
```

Code Snippet 8: Parsing date strings to datetime64 and extracting Year, Month, and DayOfWeek features (Step 10).

Step 11: Missing Value Handling. 13 rows (0.11%) with missing Age or Sex were dropped. Imputation was inappropriate for demographic fields.

This action to drop, as opposed to impute, these 13 records (.11% of 11981 total) was intentional and well-grounded methodologically. In the case of core demographic identifiers (like Age and Sex), the application of statistical imputation methods (mean imputation, k-nearest neighbours or regression-based imputation) would cause artificial individual attributes to be added to a public health dataset. A biological sex and age of a person are not variables that can be reliably deduced because unlike continuous variables like a measurement, which can be fairly approximated using correlated variables, they are facts. Imputation of Age, such as imputation using the mean age of 44 years may bias the age-based classification model systematically and artificially concentrate cases around the mean, so that the age distribution is flattened. Besides, there is no substantial effect of the insignificant loss of 13 records (0.11%) on the representativeness of the dataset: the 11,968-record dataset obtained has 99.89% of original observations, which is more than enough to make machine learning and statistical modelling robust.

```
In [14]:

1 # Step 11: Handle missing values in critical columns
2 # Issue: Age (2), Sex (12), Race (57), Date (0) have missing values
3 # Strategy: Drop rows – imputation is inappropriate for demographics
4 # (cannot reliably infer someone's age, sex, or race from other fields)
5 print("BEFORE – Missing values in key columns:")
6 key_cols = ['Age', 'Sex', 'Race', 'Date']
7 for col in key_cols:
8     print(f" {col}: {df[col].isna().sum()} missing")
9 print(f" Total rows: {len(df):,}")
10
11 df.dropna(subset=['Age', 'Sex', 'Race', 'Date'], inplace=True)
12 df.reset_index(drop=True, inplace=True)
13
14 print(f"\nAFTER – Rows remaining: {len(df):,}")
15 print(f" Rows removed: {11981 - len(df)} ({(11981 - len(df))/11981*100:.2f}%)")
16
```

Output:

```
BEFORE – Missing values in key columns:
Age: 2 missing
Sex: 12 missing
Race: 0 missing
Date: 0 missing
Total rows: 11,981

AFTER – Rows remaining: 11,968
Rows removed: 13 (0.11%)
```

Code Snippet 9: Dropping 13 rows (0.11%) with missing Age or Sex values where imputation was inappropriate (Step 11).

Step 12: Duplicate Check. Zero duplicate rows were found. No deduplication action was required.

```
In [15]:

1 # Step 12: Check for and remove duplicate records
2 duplicates = df.duplicated().sum()
3 print(f"Duplicate rows found: {duplicates}")
4
5 if duplicates > 0:
6     df.drop_duplicates(inplace=True)
7     df.reset_index(drop=True, inplace=True)
8     print(f"After removal: {len(df):,} rows remaining")
9 else:
10     print("No duplicates – no action needed.")
11
```

Output:

```
Duplicate rows found: 0
No duplicates – no action needed.
```

Code Snippet 10: Confirming zero duplicate records in the dataset (Step 12).

1.4.2 Data Transformation

Step 13: Feature Engineering — Age_Group and Drug_Count. Age_Group bins continuous age into 7 CDC-aligned categories. Drug_Count is the total substances detected per case (mean=2.34, max=9).

```
In [17]:

1 # Step 13: Feature Engineering — Age Groups and Drug Count
2 # Age groups: aligned with CDC age groupings for public health consistency
3 # Drug Count: total substances per case — useful for anomaly detection and classification
4 bins = [0, 19, 29, 39, 49, 59, 69, 100]
5 labels = ['<20', '20-29', '30-39', '40-49', '50-59', '60-69', '70+']
6 df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=True)
7
8 substance_cols_for_count = ['Heroin', 'Cocaine', 'Fentanyl', 'Fentanyl Analogue',
9                             'Oxycodone', 'Oxymorphone', 'Ethanol', 'Hydrocodone', 'Benzodiazepine',
10                            'Methadone', 'Meth/Amphetamine', 'Amphet', 'Tramad', 'Hydromorphone',
11                            'Morphine (Not Heroin)', 'Xylazine', 'Gabapentin']
12 df['Drug_Count'] = df[substance_cols_for_count].sum(axis=1)
13
14 print("Age_Group distribution:")
15 print(df['Age_Group'].value_counts().sort_index().to_string())
16 print(f"\nDrug_Count statistics:")
17 print(df['Drug_Count'].describe().to_string())
18
```

Output:

```
Age_Group distribution:
Age_Group
<20      84
20-29   1710
30-39   2974
40-49   2759
50-59   2934
60-69   1369
70+     138

Drug_Count statistics:
count    11968.000000
mean         2.342664
std         1.117921
min         0.000000
25%         2.000000
50%         2.000000
75%         3.000000
max         9.000000
```

Code Snippet 11: Engineering the Age_Group categorical variable and Drug_Count polysubstance indicator (Step 13).

Step 14: Label Encoding. Sex, Race, and Age_Group were encoded into numeric values. LabelEncoder is suitable for tree-based models; scaling is applied for distance-based models.

The reason why the labelEncoder is used instead of OneHotEncoder is due to the fact that the two algorithms that are the most sensitive to ordinal imposition in terms of consumed features are random forest and k-Means. In tree-based models like Random Forest, ordinal imposition is trivial: the algorithm subdivides the features based on threshold comparisons (e.g. Race < 2.5), and the arbitrary numerical order of the LabelEncoder has no impact on the validity of any specific split.

Any racial grouping could still be perfectly divided by a node no matter what integer label was given. In the case of distance-based algorithms like K-Means and SVM, however, LabelEncoder presents a possible artefact: defining Race=0, Race=1, Race=2 means that White numbers twice as many as Black, which cannot be given any ethnological meaning and may have perverse effects on the calculation of Euclidean distance. This limitation was reduced by using StandardScaler on all the features that are coded prior to distance-based algorithms, which standardises the scale, but not the ordinal structure. The nominal features used with the LabelEncoder were thus a purposeful tradeoff, mentioned as a remaining methodological weakness in Section 5.2, as opposed to an omission.

In [18]:

```
1 # Step 14: Encode categorical variables for ML algorithms
2 # Strategy: LabelEncoder for ordinal-safe columns
3 # Note: LabelEncoder assigns arbitrary integers – suitable for tree-based models
4 # For distance-based models (SVM, K-Means), we use scaling on top
5 from sklearn.preprocessing import LabelEncoder
6
7 le_sex = LabelEncoder()
8 df['Sex_Encoded'] = le_sex.fit_transform(df['Sex'])
9 print("Sex encoding:")
10 for i, label in enumerate(le_sex.classes_):
11     print(f" {label} → {i}")
12
13 le_race = LabelEncoder()
14 df['Race_Encoded'] = le_race.fit_transform(df['Race'])
15 print("\nRace encoding:")
16 for i, label in enumerate(le_race.classes_):
17     print(f" {label} → {i}")
18
19 le_age = LabelEncoder()
20 df['AgeGroup_Encoded'] = le_age.fit_transform(df['Age_Group'])
21 print("\nAge_Group encoding:")
22 for i, label in enumerate(le_age.classes_):
23     print(f" {label} → {i}")
24
```

Output:

```
Sex encoding:
  Female → 0
  Male → 1

Race encoding:
  American Indian or Alaska Native → 0
  Asian → 1
  Black or African American → 2
  Native Hawaiian or Other Pacific Islander → 3
  Other/Unknown → 4
  White → 5

Age_Group encoding:
  20-29 → 0
  30-39 → 1
  40-49 → 2
  50-59 → 3
  60-69 → 4
  70+ → 5
  <20 → 6
```

Code Snippet 12: Applying LabelEncoder to Sex, Race, and Age_Group for tree-based model compatibility (Step 14).

Step 15: Feature Scaling with StandardScaler. Age, Sex_Encoded, Race_Encoded, and Drug_Count were scaled to zero mean and unit variance for distance-based algorithms (SVM, K-Means).

```
In [19]:
1 # Step 15: Feature Scaling (StandardScaler)
2 # Why: Distance-based algorithms (SVM, K-Means) are sensitive to feature magnitude
3 # Age ranges from 0-100 while binary drug columns are 0-1; without scaling, Age dominates
4 # Note: Random Forest and Isolation Forest are scale-invariant – scaling not needed for them
5 from sklearn.preprocessing import StandardScaler
6
7 features_to_scale = ['Age', 'Sex_Encoded', 'Race_Encoded', 'Drug_Count']
8
9 scaler = StandardScaler()
10 scaled_data = scaler.fit_transform(df[features_to_scale])
11 df_scaled = pd.DataFrame(scaled_data, columns=[f'{col}_Scaled' for col in features_to_scale],
12                          index=df.index)
13 df = pd.concat([df, df_scaled], axis=1)
14
15 print("Scaled features (first 5 rows):")
16 print(df[[f'{col}_Scaled' for col in features_to_scale]].head().to_string())
17 print(f"\nMeans (should be ~0): {df_scaled.mean().round(4).to_dict()}")
18 print(f"\nStds (should be ~1): {df_scaled.std().round(4).to_dict()}")
19
```

Output:

```
Scaled features (first 5 rows):
   Age_Scaled  Sex_Encoded_Scaled  Race_Encoded_Scaled  Drug_Count_Scaled
0  -0.552960         0.588929         -2.384283         -1.201086
1  -0.552960         0.588929          0.421252         -1.201086
2  -1.262867         0.588929          0.421252         -1.201086
3  -1.420624        -1.697997          0.421252         -1.201086
4  -0.237446         0.588929          0.421252         -1.201086

Means (should be ~0): {'Age_Scaled': 0.0, 'Sex_Encoded_Scaled': 0.0, 'Race_Encoded_Scaled': -0.0, 'Drug_Count_Scaled': -0.0}
Stds (should be ~1): {'Age_Scaled': 1.0, 'Sex_Encoded_Scaled': 1.0, 'Race_Encoded_Scaled': 1.0, 'Drug_Count_Scaled': 1.0}
```

Code Snippet 13: Applying StandardScaler to continuous and encoded features for distance-based algorithms (Step 15).

1.4.3 Data Integration

Step 16: Geographic Coordinate Extraction. Coordinates embedded as strings were parsed using regex into separate float columns. Extraction success: 97.9% for injury, 100% for death coordinates.

```
In [21]:

1 # Step 16: Extract latitude/longitude from geocoded string fields
2 # The raw dataset stores coordinates as strings like 'CITY, ST\n(lat, lon)'
3 # Strategy: Use regex to parse numeric coordinates into separate float columns
4 def extract_coords(geo_string):
5     if pd.isna(geo_string):
6         return pd.Series([np.nan, np.nan])
7     match = re.search(r'\s*([^\s,]+),\s*([^\s,]+)\s*', str(geo_string))
8     if match:
9         try:
10             return pd.Series([float(match.group(1)), float(match.group(2))])
11         except ValueError:
12             return pd.Series([np.nan, np.nan])
13     return pd.Series([np.nan, np.nan])
14
15 df[['Injury_Lat', 'Injury_Lon']] = df['InjuryCityGeo'].apply(extract_coords)
16 df[['Death_Lat', 'Death_Lon']] = df['DeathCityGeo'].apply(extract_coords)
17
18 print(f"Injury coordinates extracted: {df['Injury_Lat'].notna().sum():,} / {len(df):,} "
19       f"f"({df['Injury_Lat'].notna().mean()*100:.1f}%)"
20 print(f"Death coordinates extracted: {df['Death_Lat'].notna().sum():,} / {len(df):,} "
21       f"f"({df['Death_Lat'].notna().mean()*100:.1f}%)"
22
```

Output:

```
Injury coordinates extracted: 11,712 / 11,968 (97.9%)
Death coordinates extracted: 11,967 / 11,968 (100.0%)
```

Code Snippet 14: Parsing geocoded coordinate strings into separate latitude and longitude float columns using regex (Step 16).

1.4.4 Data Reduction

Step 17: Remove Redundant Columns. 12 columns were removed for reasons including redundancy with extracted coordinates, >90% missing values, free-text content, and zero variance.

Each of the four groups of removed columns can be seen as a methodological justification. To begin with, geocoded columns of coordinate (e.g., "Injury Location" in a single string field) were deleted since Step 16 had already retrieved their latitude and longitude values as individual, machine-readable float columns. Storing the original string representations would add pure redundancy by having them both, with the floats that have been read out, which might be confusing to the distance based algorithms which might be operating on the resultant features. Second, columns where more than 90% of the values were missing (like some of the secondary substance fields) were dropped since no imputation strategy can identify data that has been almost entirely lost; including such columns would add large amounts of data that are created artificially to the feature set. Thirdly, and most importantly to accommodate machine learning, all the free-text narrative columns including the description of injury and the cause of death were eliminated. These domains hold prose in any form of unstructured natural language that cannot be directly absorbed

by any of the four data mining algorithms used in this project. They would need Natural Language Processing (NLP) methods, such as tokenisation, term frequency-inverse document frequency (TF-IDF) vectorisation, and potentially transformer-based embeddings, which would be beyond the scope of the specified project. Leaving them in their original form would make the panda to consider them as object-type features and either they fail during the model fitting or they encode them in an arbitrary way. Fourth, columns with zero variance (those with one repeated value on all 11,981 records) were dropped since they have no discriminative value whatsoever; an attribute with which all cases take the same value cannot form part of any classification boundary, regression coefficient or cluster centroid.

In [23]:

```
1 # Step 17: Remove columns that are redundant, high-missingness, or out of scope
2 columns_to_drop = {
3     'ResidenceCityGeo': 'Replaced by extracted coordinates',
4     'InjuryCityGeo': 'Replaced by extracted Injury_Lat, Injury_Lon',
5     'DeathCityGeo': 'Replaced by extracted Death_Lat, Death_Lon',
6     'Location if Other': '90% missing values – provides no statistical value',
7     'Other Significant Conditions': '90% missing values',
8     'Description of Injury': 'Free-text field – NLP analysis is out of scope',
9     'Cause of Death': 'Free-text field – NLP analysis is out of scope',
10    'Date Type': 'Single constant value (Date of Death) – zero variance',
11    'Heroin death certificate (DC)': 'Redundant with Heroin detection column',
12    'Opiate NOS': 'Very low detection count – insufficient for pattern mining',
13    'Heroin/Morph/Codeine': 'Redundant composite – individual drug columns retained',
14    'Other Opioid': 'Ambiguous category – not useful for specific drug analysis'
15 }
16
17 print("Columns removed and justification:")
18 for col, reason in columns_to_drop.items():
19     if col in df.columns:
20         print(f" x {col}: {reason}")
21
22 df.drop(columns=[c for c in columns_to_drop if c in df.columns], inplace=True)
23 print(f"\nDataset shape after reduction: {df.shape[0]:,} rows x {df.shape[1]} columns")
24
```

Output:

```
Columns removed and justification:
 x ResidenceCityGeo: Replaced by extracted coordinates
 x InjuryCityGeo: Replaced by extracted Injury_Lat, Injury_Lon
 x DeathCityGeo: Replaced by extracted Death_Lat, Death_Lon
 x Location if Other: 90% missing values – provides no statistical value
 x Other Significant Conditions: 90% missing values
 x Description of Injury: Free-text field – NLP analysis is out of scope
 x Cause of Death: Free-text field – NLP analysis is out of scope
 x Date Type: Single constant value (Date of Death) – zero variance
 x Heroin death certificate (DC): Redundant with Heroin detection column
 x Opiate NOS: Very low detection count – insufficient for pattern mining
 x Heroin/Morph/Codeine: Redundant composite – individual drug columns retained
 x Other Opioid: Ambiguous category – not useful for specific drug analysis

Dataset shape after reduction: 11,968 rows x 52 columns
```

Code Snippet 15: Removing 12 redundant, high-missing, free-text, and zero-variance columns (Step 17).

1.4.5 Preprocessing Summary

The preprocessing pipeline transformed the dataset from 11,981 rows $\times$ 48 columns to 11,968 rows $\times$ 52 columns. Only 13 rows (0.11%) were removed. 12 columns were dropped and 16 new ones added.

In [25]:

```
1 # Step 18: Final preprocessing summary
2 print("=" * 60)
3 print("PREPROCESSING SUMMARY")
4 print("=" * 60)
5 print(f"\n Original dataset:      11,981 rows x 48 columns")
6 print(f"  Preprocessed dataset: {df.shape[0]:,} rows x {df.shape[1]} columns")
7 print(f"    Rows removed:          {11981 - df.shape[0]} (missing critical fields)")
8 print(f"\nData types breakdown:")
9 print(df.dtypes.value_counts().to_string())
10 print(f"\nRemaining missing values in key analysis columns:")
11 for col in ['Age', 'Sex', 'Race', 'Year', 'Fentanyl', 'Any Opioid', 'Drug_Count']:
12     print(f"  {col}: {df[col].isna().sum()}")
13
14 # Export cleaned dataset
15 df.to_csv('Cleaned_Group_Dataset.csv', index=False)
16 print(f"\nExported: Cleaned_Group_Dataset.csv")
17
```

Output:

```
=====
PREPROCESSING SUMMARY
=====

Original dataset:      11,981 rows x 48 columns
Preprocessed dataset:  11,968 rows x 52 columns
Rows removed:         13 (missing critical fields)

Data types breakdown:
int64          23
str            15
float64         9
int32           3
datetime64[us]  1
category        1

Remaining missing values in key analysis columns:
Age: 0
Sex: 0
Race: 0
Year: 0
Fentanyl: 0
Any Opioid: 0
Drug_Count: 0

Exported: Cleaned_Group_Dataset.csv
```

Code Snippet 16: Final preprocessing pipeline summary: 11,981 $\rightarrow$ 11,968 rows, 48 $\rightarrow$ 52 columns.

2. Implementation of Data Mining Techniques

This part records the process of four data mining methods. Two algorithms are compared against each other with full justification, code documentation and results.

2.1 Classification: Random Forest vs. Support Vector Machine

Problem Under Review: Problem 1 - Opioid Involvement Prediction. Target: Any Opioid (binary: 1 = opioid involved, 0 = not).

2.1.1 Why Classification?

Classification suits well as the issue demands a discrete binary answer. Regression would be used to forecast continuous values thus losing the categorical difference. Clustering would have grouped cases but will not be able to make predictions. This problem structure is not applicable to either of the alternatives.

More specifically, classification is suitable not only due to the fact that the outcome variable is a dichotomous one, but also due to the fact that the downstream decision represented by that outcome variable is also a dichotomous one, namely, whether a public health planner or clinician will direct naloxone resources, treatment referrals or outreach capacity to a specific case profile or not. The result of the regression, as in the case of 0.73 probability of opioid involvement, cannot be directly applied in this case because it needs an arbitrary decision threshold before it can be used to act on any resource allocation. A classification model that trains the best boundary using the data trains feature combinations directly to an intervention / no-intervention outcome and the threshold decision is thus embedded in the model instead of being pushed off. The fact that classification is used instead of regression is hence a reflection of the structure of the real-world decision under support and not the structure of the target variable.

2.1.2 Why Random Forest?

Random Forest accepts mixed types of features without normalisation, offers intrinsic feature importance scores and is resistant to overfitting with ensemble averaging. It is non-parametric and

has no distribution assumptions. One Decision Tree was discarded based on tendency of overfitting; Naive Bayes was discarded as it assumes independence of the features, which correlated drug indicators violate.

The core strength of the Random Forest method lies in its ensemble architecture: many decision trees are constructed, each trained using a random bootstrap sample of the data and only a random sample of the features at each split. Prediction is done by majority vote through all trees. This dual randomisation ensures that none of the trees has memorised the training data so that the variance issue that plagues any single decision tree is greatly dampened. It is particularly useful in this dataset since the 10 input features contain some correlated drug indicators (e.g., Fentanyl and Xylazine are positively related) - an individual tree would overfit to them, but a forest would average them out. Another part of the analysis output that was interesting (though not quite as important to us as the first two) was the built-in feature importance measure, which is based on the mean decrease in Gini impurity of any split, but it again answers our research question of what contributes most to distinguishing opioid and non-opioid cases: Age (0.33), Drug_Count (0.25), and Xylazine (0.13). Instead of default hyperparameters, GridSearchCV hyperparameter optimization was required since the hyperparameters of importance - `n_estimators` (number of trees), `max_depth` (depth limit of a tree), `min_samples_split` (records needed to split a node) - directly and non-trivially affect the bias-variance tradeoff. With default values that would be to accept an arbitrary configuration as opposed to the one that is empirically demonstrated to be maximising F1 score on this particular dataset.

2.1.3 Why SVM as Comparison?

SVM is a radically new paradigm- a margin based classifier instead of a tree based ensemble. It optimises the margin between classes on the hyperplane and works well in high dimension spaces. With the RBF kernel, non-linear decision boundaries can be used to represent heterogeneous patterns of drug interactions.

Where the decision boundary in a random forest is created by a democratic vote of the multiple simple trees, the single geometrically optimal boundary created by the SVM is the hyperplane that maximises the distance (margin) between the nearest data points of each of the two classes, the

support vectors. This maximum-margin feature renders SVM conceptually resistant to overfitting, especially when the dimension of the feature space is large. Radial Basis Function (RBF) kernel was chosen due to its implicit transformation of the 10-dimensional feature space to an infinite-dimensional space to enable SVM to adopt curved, non-linear decision boundaries, which can be used to model the interaction between drug indicators and demographic variables that might go unnoticed by a linear boundary. Designing the SVM pipeline as StandardScaler-SVC-Pipeline wrapping the Pipeline object provided by sklearn was methodologically correct: it can be guaranteed that the scaler is never fit to test fold data, and it can be guaranteed that the scaler is never fit again within an iteration, thus avoiding the effect of data leaking into the overall performance evaluation. The two important SVM hyperparameters, C (regularisation parameter that balances the desire to maximise the margin and the desire to minimise training errors) and gamma (influence radius of individual support vectors when using the RBF kernel) needed to be optimised with the help of GridSearchCV.

2.1.4 Data Preparation

Ten characteristics were chosen deliberately avoiding individual opioid drugs in order to avoid data leakage. There was an imbalance in the class of the target 74.7% opioid vs 25.3% non-opioid. An 80/20 stratified partition was used.

In [28]:

```
1 # 2.1.4 – Data Preparation for Classification
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.svm import SVC
6 from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold, GridSearchCV
7 from sklearn.metrics import (classification_report, accuracy_score,
8                               confusion_matrix, f1_score, roc_auc_score, roc_curve)
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.pipeline import Pipeline
11
12 # Load cleaned dataset
13 df = pd.read_csv('Cleaned_Group_Dataset.csv')
14
15 # Define features and target
16 # IMPORTANT: We deliberately EXCLUDE individual opioid drugs (Heroin, Fentanyl, Oxycodone,
17 # Methadone, etc.) from features to avoid DATA LEAKAGE – these directly define the target.
18 # We keep non-opioid substances as they are genuine predictors, not definitional.
19 feature_cols = ['Age', 'Sex_Encoded', 'Race_Encoded',
20                 'Cocaine', 'Ethanol', 'Benzodiazepine',
21                 'Meth/Amphetamine', 'Xylazine', 'Gabapentin', 'Drug_Count']
22 target = 'Any Opioid'
23
24 X = df[feature_cols]
25 y = df[target]
26
27 print(f"Feature matrix shape: {X.shape}")
28 print(f"\nTarget distribution:")
29 print(f" Opioid-involved (1): {y.sum():,} ({y.mean()*100:.1f}%)"
30 print(f" Non-opioid (0):      {(y==0).sum():,} ({(1-y.mean())*100:.1f}%)"
31
32 # Stratified train-test split (80/20) to preserve class balance
33 X_train, X_test, y_train, y_test = train_test_split(
34     X, y, test_size=0.2, random_state=42, stratify=y)
35
36 print(f"\nTraining set: {X_train.shape[0]:,} samples")
37 print(f"Test set:      {X_test.shape[0]:,} samples")
38
```

Output:

```
Feature matrix shape: (11968, 10)

Target distribution:
 Opioid-involved (1): 8,937 (74.7%)
 Non-opioid (0):      3,031 (25.3%)

Training set: 9,574 samples
Test set:      2,394 samples
```

Code Snippet 17: Feature selection and stratified 80/20 train-test split for the classification task.

2.1.5 Random Forest with GridSearchCV

GridSearchCV with 5-fold stratified CV optimised F1 score. `class_weight='balanced'` addressed class imbalance. Best parameters: `max_depth=15`, `min_samples_split=10`, `n_estimators=100`. Best CV F1: 0.7777, test accuracy: 0.6855. Age (0.3325) was the strongest predictor, followed by Drug_Count (0.2513) and Xylazine (0.1253).

In [29]:

```
1 # 2.1.5 - Random Forest with Hyperparameter Tuning (GridSearchCV)
2 # We use GridSearchCV to find the best combination of hyperparameters
3 # class_weight='balanced' addresses the class imbalance (74.6% opioid vs 25.4% non-opioid)
4 # Without this, the model would be biased toward predicting the majority class (opioid)
5
6 param_grid_rf = {
7     'n_estimators': [100, 200],
8     'max_depth': [10, 15, 20],
9     'min_samples_split': [5, 10]
10 }
11
12 rf_grid = GridSearchCV(
13     RandomForestClassifier(random_state=42, n_jobs=-1, class_weight='balanced'),
14     param_grid_rf,
15     cv=5, # 5-fold cross-validation
16     scoring='f1', # Optimise for F1 score
17     n_jobs=-1,
18     verbose=0
19 )
20
21 rf_grid.fit(X_train, y_train)
22 rf_model = rf_grid.best_estimator_
23 rf_pred = rf_model.predict(X_test)
24
25 print("=" * 60)
26 print("RANDOM FOREST - HYPERPARAMETER TUNING RESULTS")
27 print("=" * 60)
28 print(f" Note: class_weight='balanced' used to address class imbalance")
29 print(f" (Opioid: {y.mean()*100:.1f}% vs Non-Opioid: {(1-y.mean())*100:.1f}%)")
30 print(f"\nBest parameters: {rf_grid.best_params_}")
31 print(f"Best CV F1 score: {rf_grid.best_score_:.4f}")
32 print(f"\nTest Set Results:")
33 print(f" Accuracy: {accuracy_score(y_test, rf_pred):.4f}")
34 print(f"\nClassification Report:")
35 print(classification_report(y_test, rf_pred, target_names=['Non-Opioid', 'Opioid']))
36
37 # Feature importance
38 importance_df = pd.DataFrame({
39     'Feature': feature_cols,
40     'Importance': rf_model.feature_importances_
41 }).sort_values('Importance', ascending=False)
42
43 print("\nFeature Importance Ranking:")
44 for _, row in importance_df.iterrows():
45     bar = '█' * int(row['Importance'] * 50)
46     # ... (continued)
```

Output:

```
=====
RANDOM FOREST - HYPERPARAMETER TUNING RESULTS
=====
Note: class_weight='balanced' used to address class imbalance
(Opioid: 74.7% vs Non-Opioid: 25.3%)

Best parameters: {'max_depth': 15, 'min_samples_split': 10, 'n_estimators': 100}
Best CV F1 score: 0.7777

Test Set Results:
Accuracy: 0.6855

Classification Report:
              precision    recall  f1-score   support

Non-Opioid      0.42       0.60       0.49        606
Opioid          0.84       0.71       0.77       1788

 accuracy          0.69       0.69       0.69       2394
 macro avg         0.63       0.66       0.63       2394
 weighted avg      0.73       0.69       0.70       2394


Feature Importance Ranking:
Age                0.3325 ██████████
Drug_Count         0.2513 ██████████
Xylazine           0.1253 ████████
Cocaine            0.0827 ████████
Ethanol            0.0552 ████████
Race_Encoded       0.0456 ████████
Benzodiazepine     0.0393 ████████
Gabapentin         0.0315 ████████
Sex_Encoded        0.0240 ████████
Meth/Amphetamine  0.0127 ████████
```

Code Snippet 18: Random Forest classifier with GridSearchCV hyperparameter optimisation (best CV F1: 0.7777).

2.1.6 SVM with GridSearchCV

A Pipeline (StandardScaler + SVC) prevented data leakage during CV. Best parameters: C=10.0, gamma='scale', kernel='rbf'. Best CV F1: 0.7516, test accuracy: 0.6612. SVM achieved higher precision (0.86) but lower recall (0.66) than RF.

In [30]:

```
1 # 2.1.6 – SVM with Hyperparameter Tuning (GridSearchCV)
2 # SVM requires scaled features – Pipeline ensures scaling within CV (no data leakage)
3 # class_weight='balanced' adjusts penalties inversely proportional to class frequency
4
5 param_grid_svm = {
6     'svm__C': [0.1, 1.0, 10.0],
7     'svm__gamma': ['scale', 'auto'],
8     'svm__kernel': ['rbf']
9 }
10
11 svm_pipeline = Pipeline([
12     ('scaler', StandardScaler()),
13     ('svm', SVC(random_state=42, probability=True, class_weight='balanced'))
14 ])
15
16 svm_grid = GridSearchCV(
17     svm_pipeline,
18     param_grid_svm,
19     cv=5,
20     scoring='f1',
21     n_jobs=-1,
22     verbose=0
23 )
24
25 svm_grid.fit(X_train, y_train)
26 svm_model = svm_grid.best_estimator_
27 svm_pred = svm_model.predict(X_test)
28
29 print("=" * 60)
30 print("SVM – HYPERPARAMETER TUNING RESULTS")
31 print("=" * 60)
32 print(f" Note: class_weight='balanced' used to address class imbalance")
33 print(f"\nBest parameters: {svm_grid.best_params_}")
34 print(f"Best CV F1 score: {svm_grid.best_score_:.4f}")
35 print(f"\nTest Set Results:")
36 print(f" Accuracy: {accuracy_score(y_test, svm_pred):.4f}")
37 print(f"\nClassification Report:")
38 print(classification_report(y_test, svm_pred, target_names=['Non-Opioid', 'Opioid']))
```

Output:

```
=====
SVM – HYPERPARAMETER TUNING RESULTS
=====
Note: class_weight='balanced' used to address class imbalance

Best parameters: {'svm__C': 10.0, 'svm__gamma': 'scale', 'svm__kernel': 'rbf'}
Best CV F1 score: 0.7516

Test Set Results:
Accuracy: 0.6612

Classification Report:

```

	precision	recall	f1-score	support
Non-Opioid	0.40	0.68	0.50	606
Opioid	0.86	0.66	0.74	1788
accuracy			0.66	2394
macro avg	0.63	0.67	0.62	2394
weighted avg	0.74	0.66	0.68	2394

Code Snippet 19: SVM classifier with Pipeline (StandardScaler + SVC) and GridSearchCV (best CV F1: 0.7516).

2.1.7 Confusion Matrix Comparison

Figure 1 shows side-by-side confusion matrices. Random Forest achieved better balance between classes, correctly identifying more opioid cases (higher recall).

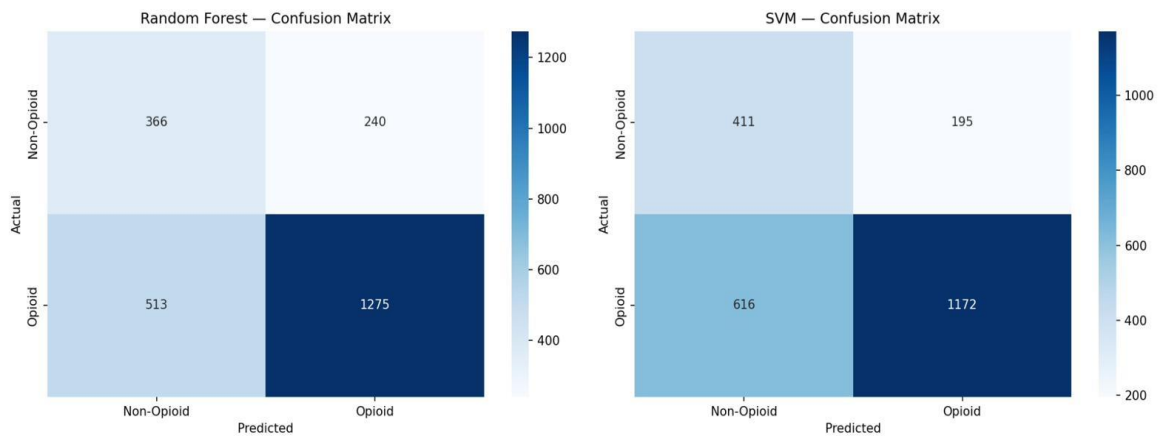


Figure 1: Confusion matrices comparing Random Forest (left) and SVM (right).

2.2 Prediction: Linear vs. Polynomial Regression

Problem Addressed: Problem 2 - Drug Overdose Substance Trends Modelling. Goal: Annual Fentanyl, Heroin, Cocaine, and Xylazine detection.

2.2.1 Why Regression?

Regression will suit the continuous numeric values (yearly counts) which are to be predicted. Time-series (ARIMA) were discarded since they need a test of stationarity - usually by an Augmented Dickey-Fuller (ADF) unit root test - prior to specification. The ADF test is based on a mean of 12 observations a year, which is too small to have a reliable chance of differentiating between a true unit root (non-stationary process in need of differencing) and a stationary process with a structural break or a strong trend. The 12-data-point limit has no ad hoc basis but is a fundamental statistical power issue: it is not possible to estimate an effective ARIMA(p,d,q) model when the observations are only slightly more than the parameters under estimation. Five hundred and preferably a thousand observations are necessary to identify an ARIMA model, and seasonally-adjusted models require at least 500 and ideally 1000 observations.

2.2.2 Why Linear Regression?

Linear Regression gives understandable slope coefficients of the precise rate of change per year and can be used as a baseline. The reason why decision tree regression was dismissed was that a tree would severely overfit with 12 data points.

2.2.3 Why Polynomial Regression (Degree 2)?

Polynomial Regression is able to pick non-linear trends like the rise-then-decline pattern of heroin. The choice of Degree 2 was motivated by the fact that only single terms of curvature (the quadratic) are needed to capture the inverted-U and J-curve shapes found in this data without excessive exposure of this data to overfitting at higher degrees of the polynomials. The 3 and higher degree was not accepted because of two reasons. To begin with, when using a limited number of data points, such as 12, the bias-variance trade-off is very unfavourable: a degree-6 polynomial, e.g. can fit all 12 points (interpolating instead of generalising) with almost zero training error, but predicting disastrously with 12 data points. Second, high-degree polynomial interpolation is prone

to the Runge phenomenon that is high-degree polynomials tend to wiggle and go to extremes between data points, especially at the edges of the fitted range. A degree-5 (or greater) time series would give a time series that would be oscillating the observed annual values to extreme values, making any predictions based on that very time series statistically invalid, not to mention physically invalid (e.g., negative counts of deaths in projected years). Degree 2 prevents either of these failure cases and still represents the important non-linear trends in the data.

2.2.4 Yearly Aggregated Data

```
In [33]:
1 # 2.2.4 – Prepare yearly aggregated data
2 from sklearn.linear_model import LinearRegression
3 from sklearn.preprocessing import PolynomialFeatures
4 from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
5
6 key_drugs = ['Fentanyl', 'Heroin', 'Cocaine', 'Xylazine']
7 yearly_data = df.groupby('Year')[key_drugs].sum().reset_index()
8
9 print("Yearly substance detection counts:")
10 print(yearly_data.to_string(index=False))
11
```

Output:

Year	Fentanyl	Heroin	Cocaine	Xylazine
2012	14	174	105	0
2013	37	257	147	0
2014	74	325	127	0
2015	189	417	177	0
2016	482	494	275	0
2017	675	474	347	0
2018	760	391	345	0
2019	978	386	462	0
2020	1155	262	529	139
2021	1301	166	655	301
2022	1252	125	683	350
2023	1122	106	720	286

Code Snippet 20: Aggregating annual substance detection counts for regression analysis (2012–2023).

2.2.5 Linear Regression Results

Fentanyl: +133.3 cases/year, $R^2=0.9295$ (strong linear increase). Heroin: −14.8 cases/year, $R^2=0.1540$ (linear model fails—non-linear trajectory). Cocaine: +61.6 cases/year, $R^2=0.9668$ (steady increase). Xylazine: +31.8 cases/year, $R^2=0.6646$ (moderate fit).

In [34]:

```
1 # 2.2.5 – Linear Regression for each substance
2 print("=" * 65)
3 print("LINEAR REGRESSION – SUBSTANCE TREND PREDICTION")
4 print("=" * 65)
5
6 linear_results = {}
7 linear_models = {}
8 for drug in key_drugs:
9     X_year = yearly_data[['Year']].values
10    y_counts = yearly_data[drug].values
11
12    model = LinearRegression()
13    model.fit(X_year, y_counts)
14    y_pred = model.predict(X_year)
15
16    r2 = r2_score(y_counts, y_pred)
17    mae = mean_absolute_error(y_counts, y_pred)
18    rmse = np.sqrt(mean_squared_error(y_counts, y_pred))
19    slope = model.coef_[0]
20    direction = "INCREASING" if slope > 0 else "DECREASING"
21
22    linear_results[drug] = {'R2': r2, 'MAE': mae, 'RMSE': rmse, 'slope': slope}
23    linear_models[drug] = model
24
25    print(f"\n {drug}:")
26    print(f"    Trend:    {direction} ({slope:+.1f} cases/year)")
27    print(f"    R²:       {r2:.4f}")
28    print(f"    MAE:      {mae:.1f}")
29    print(f"    RMSE:     {rmse:.1f}")
30
```

Output:

```
=====
LINEAR REGRESSION – SUBSTANCE TREND PREDICTION
=====

Fentanyl:
  Trend:    INCREASING (+133.3 cases/year)
  R²:       0.9295
  MAE:      101.5
  RMSE:     126.7

Heroin:
  Trend:    DECREASING (-14.8 cases/year)
  R²:       0.1540
  MAE:      105.9
  RMSE:     119.7

Cocaine:
  Trend:    INCREASING (+61.6 cases/year)
  R²:       0.9668
  MAE:      31.6
  RMSE:     39.4

Xylazine:
  Trend:    INCREASING (+31.8 cases/year)
  R²:       0.6646
  MAE:      66.5
  RMSE:     78.0
```

Code Snippet 21: Linear Regression fitted to yearly substance trends (Fentanyl R^2 : 0.9295, Heroin R^2 : 0.1540).

2.2.6 Polynomial Regression Results and Comparison

Polynomial outperformed linear for all substances. Heroin: R^2 jumped $0.1540 \rightarrow 0.8730$ (biggest improvement—quadratic captures rise-and-fall). Fentanyl: $0.9295 \rightarrow 0.9409$. Cocaine: $0.9668 \rightarrow 0.9796$. Xylazine: $0.6646 \rightarrow 0.8568$.

In [35]:

```
1 # 2.2.6 – Polynomial Regression (degree=2) with visualisation
2 print("=" * 65)
3 print("POLYNOMIAL REGRESSION (DEGREE 2) – SUBSTANCE TREND PREDICTION")
4 print("=" * 65)
5
6 poly = PolynomialFeatures(degree=2, include_bias=False)
7 poly_results = {}
8
9 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
10 axes = axes.ravel()
11
12 for idx, drug in enumerate(key_drugs):
13     X_year = yearly_data[['Year']].values
14     y_counts = yearly_data[drug].values
15
16     # Polynomial model
17     X_poly = poly.fit_transform(X_year)
18     model_poly = LinearRegression()
19     model_poly.fit(X_poly, y_counts)
20     y_pred_poly = model_poly.predict(X_poly)
21
22     # Linear predictions for comparison
23     y_pred_lin = linear_models[drug].predict(X_year)
24
25     r2_poly = r2_score(y_counts, y_pred_poly)
26     mae_poly = mean_absolute_error(y_counts, y_pred_poly)
27     rmse_poly = np.sqrt(mean_squared_error(y_counts, y_pred_poly))
28
29     poly_results[drug] = {'R2': r2_poly, 'MAE': mae_poly, 'RMSE': rmse_poly}
30
31     print(f"\n {drug}:")
32     print(f"    R² (Poly):    {r2_poly:.4f} vs R² (Linear): {linear_results[drug]['R2']:.4f}")
33     print(f"    MAE (Poly):    {mae_poly:.1f} vs MAE (Linear): {linear_results[drug]['MAE']:.1f}")
34
35     # Plot
36     ax = axes[idx]
37     ax.scatter(yearly_data['Year'], y_counts, color='black', s=60, zorder=5, label='Actual')
38     ax.plot(yearly_data['Year'], y_pred_lin, '--', color='steelblue', linewidth=2, label=f'Linear (R²={li
39     near_results[drug]["R2"]:.3f})')
40     ax.plot(yearly_data['Year'], y_pred_poly, '-', color='crimson', linewidth=2, label=f'Poly d=2 (R²={r2
41     _poly:.3f})')
42     ax.set_title(f'{drug} – Yearly Trend')
43     ax.set_xlabel('Year')
44     ax.set_ylabel('Count')
45     ax.legend(fontsize=8)
46     # ... (continued)
```

Output:

```
=====
POLYNOMIAL REGRESSION (DEGREE 2) – SUBSTANCE TREND PREDICTION
=====

Fentanyl:
  R² (Poly):    0.9409 vs R² (Linear): 0.9295
  MAE (Poly):    93.8 vs MAE (Linear): 101.5

Heroin:
  R² (Poly):    0.8730 vs R² (Linear): 0.1540
  MAE (Poly):    38.6 vs MAE (Linear): 105.9

Cocaine:
  R² (Poly):    0.9796 vs R² (Linear): 0.9668
  MAE (Poly):    26.4 vs MAE (Linear): 31.6

Xylazine:
  R² (Poly):    0.8568 vs R² (Linear): 0.6646
  MAE (Poly):    38.4 vs MAE (Linear): 66.5
<Figure size 1400x1000 with 4 Axes>
```

Code Snippet 22: Polynomial Regression (degree 2) fitted to substance trends (Heroin R^2 improved: 0.15
→ 0.87).

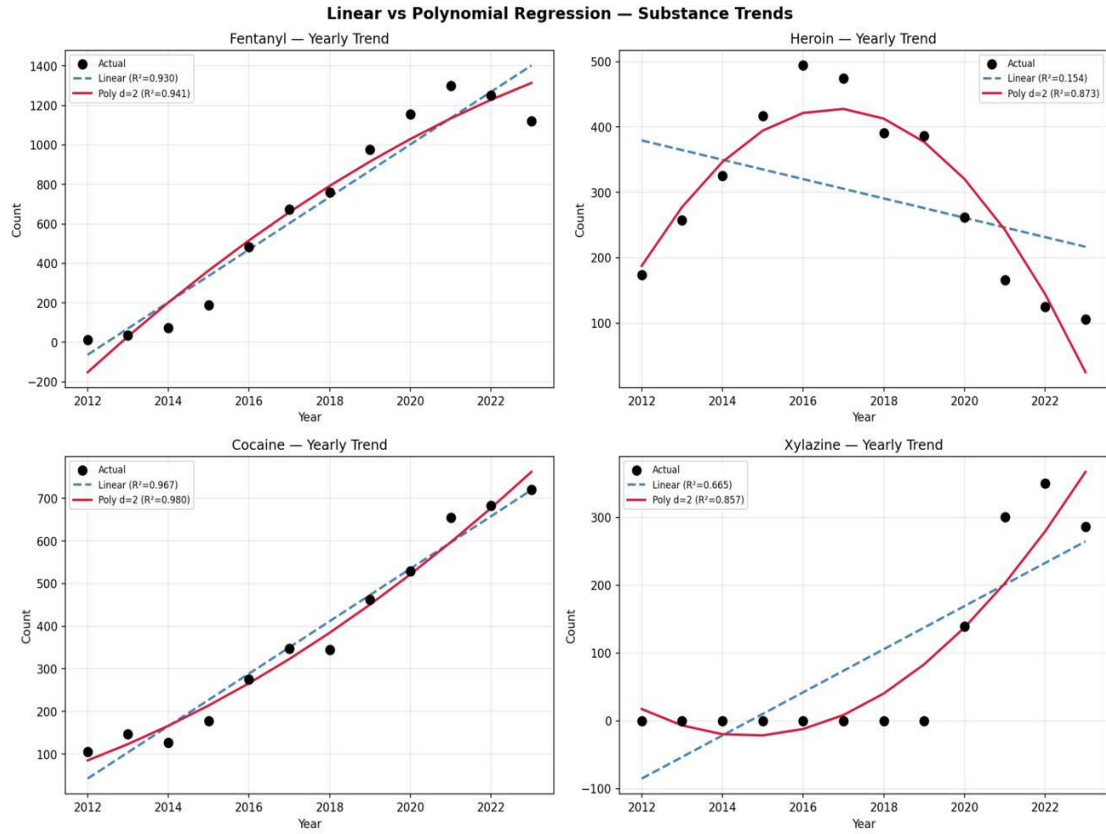


Figure 2: Linear vs. Polynomial Regression comparison for four substances.

2.3 Clustering: K-Means vs. DBSCAN

Problem Addressed: Problem 3 – Identifying Geographic Clusters. **Features:** Injury_Lat, Injury_Lon.

2.3.1 Why Clustering?

Clustering finds some natural groupings in geographic data without predetermined labels. Clustering inherently extracts structure in unlabelled spatial data as opposed to classification (use of labels required) or regression (prediction of values).

Most importantly, the use of clustering in this case would be a correct approach since we are not informed by the prevailing geographical hotspots, and applying a defined spatial border - say the eight counties of Connecticut or 169 towns - would merely prove the existence of administrative boundaries and not the places of concentration of deaths. Clustering enables the data to determine the geographic structure by letting it be defined by the data itself, which is where it is most densely populated by overdose and where it is least densely populated by overdose. This is operationally important: cluster around the centre of Hartford that the algorithm attracts can cross municipal boundaries to include neighbouring municipal towns with the same drug supply network, which a county approach would artificially isolate. This is something that cannot be done by classification since there is no pre-labelled hotspot and non-hotspot training available; or by regression since spatial concentration is not an interval value that must be predicted. The only method that considers geographic structure as a discovery and not an assumption is clustering.

2.3.2 Why K-Means?

Geographic data create approximately spheric clusters around city centres- K-Means is good at this. It can support 11,000 or more records, and the interpretable centroids can be mapped to actual locations. Hierarchical clustering was discarded as it is more expensive than $O(n^2)$; GMM was discarded due to the possibility of overfitting on just 2 features.

K-Means has a centroid-like principle: it assumes that clusters are approximately spherical, compact and similarly sized, and assigns each data point to the closest centroid and updates centroid as the average of points assigned to it, and repeats until convergence. This assumption is natively geographically appropriate to Connecticut. The mortality due to drug overdose in the state is also clustered in its cities - Hartford, New Haven, Bridgeport, Waterbury, New London - each of which is a gravitational centre and is flanked by the shrinking suburban and rural population. The concentration of urban deaths is similar to the geometry of a cluster of stars that K-Means is supposed to identify, so it will be a natural option to start with this issue. The Silhouette analysis to decide on the best k ($k=11$) was not an option but a necessity: in contrast to supervised algorithms where the number of classes is predetermined by the data, unsupervised clustering must be specified by the analyst on the amount of clusters, and the selection of k essentially creates or removes the geographic regions in the data. The Silhouette score which is the measure of the similarity of each point to its corresponding cluster against neighboring clusters gave an objective measure of the correct choice of $k=11$ instead of arbitrary one.

2.3.3 Why DBSCAN?

DBSCAN finds arbitrarily-shaped clusters, automatically classifies noise (isolated rural deaths) and does not need any prior-specified k . A comparison between centroid-based (K-Means) and density-based (DBSCAN) shows which of the two assumptions is more fitting to the geographic distribution.

In contrast to K-Means where compact spherical clusters are assumed, DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is based on a completely different principle, creating clusters as dense areas between sparse ones. A core point is defined to be a point that has `min_samples` neighbours within a radius of `eps`; clusters are created by joining core points together, and all the points that cannot be reached by a core point are considered to be noise. The difference has a great practical importance to the geography of overdose in Connecticut. The drug death environment in Connecticut is not evenly distributed: the urban centres, such as Hartford, are

densely populated in terms of the number of deaths compared to the rural setting in such counties as Windham and Tolland. K-Means must mark all the rural deaths using the closest city centroid artificially bloating those clusters. DBSCAN, in its turn, can be genuinely honest about such rural deaths, which can be seen as noise (marked -1), which are isolated cases which are not a part of any regional hot spot. These 1,323 noise points detected by DBSCAN at Phase 2 are not an algorithmic failure - they will be a true geographic discovery: those rural deaths that must be approached by mobile outreach programs, quite unlike those in urban intervention programmes. This complementary view was the main factor of why DBSCAN was included with K-Means instead of applying one only.

2.3.4 Data Preparation

Missing coordinates were screened out, kept within the state of Connecticut. 11,712 records were retained. StandardScaler was used to scale coordinates.

```
In [37]:
1 # 2.3.4 – Data Preparation for Clustering
2 from sklearn.cluster import KMeans, DBSCAN
3 from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
4
5 # Prepare geographic data (drop rows with missing coordinates)
6 geo_df = df.dropna(subset=['Injury_Lat', 'Injury_Lon']).copy()
7
8 # Filter to Connecticut bounds to remove geocoding errors
9 geo_df = geo_df[(geo_df['Injury_Lat'] > 40.9) & (geo_df['Injury_Lat'] < 42.1) &
10                (geo_df['Injury_Lon'] > -73.8) & (geo_df['Injury_Lon'] < -71.7)]
11
12 print(f"Geographic records after filtering: {len(geo_df):,}")
13
14 # Scale coordinates – important for K-Means distance calculations
15 geo_features = geo_df[['Injury_Lat', 'Injury_Lon']].values
16 scaler_geo = StandardScaler()
17 geo_scaled = scaler_geo.fit_transform(geo_features)
18
```

Output:

Geographic records after filtering: 11,712

Code Snippet 23: Filtering to valid Connecticut coordinates and standardising geographic features for clustering.

2.3.5 K-Means: Optimal k Selection

Silhouette analysis identified k=11 as optimal (highest Silhouette=0.5689).

In [38]:

```
1 # 2.3.5 – K-Means: Determine optimal k using Elbow + Silhouette Analysis
2 k_range = range(3, 12)
3 silhouette_scores = []
4 inertias = []
5
6 for k in k_range:
7     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
8     labels = kmeans.fit_predict(geo_scaled)
9     silhouette_scores.append(silhouette_score(geo_scaled, labels))
10    inertias.append(kmeans.inertia_)
11    print(f" k={k}: Silhouette={silhouette_scores[-1]:.4f}, Inertia={inertias[-1]:.1f}")
12
13 optimal_k = list(k_range)[np.argmax(silhouette_scores)]
14 print(f"\nOptimal k: {optimal_k} (Silhouette={max(silhouette_scores):.4f})")
15
16 # Plot
17 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
18
19 axes[0].plot(list(k_range), inertias, 'bo-')
20 axes[0].set_title('Elbow Method – Inertia vs k')
21 axes[0].set_xlabel('Number of Clusters (k)')
22 axes[0].set_ylabel('Inertia (Within-Cluster Sum of Squares)')
23 axes[0].grid(True, alpha=0.3)
24
25 axes[1].plot(list(k_range), silhouette_scores, 'ro-')
26 axes[1].set_title('Silhouette Analysis – Score vs k')
27 axes[1].set_xlabel('Number of Clusters (k)')
28 axes[1].set_ylabel('Silhouette Score')
29 axes[1].axvline(x=optimal_k, color='green', linestyle='--', label=f'Optimal k={optimal_k}')
30 axes[1].legend()
31 axes[1].grid(True, alpha=0.3)
32
33 plt.tight_layout()
34 plt.savefig('clustering_optimal_k.png', dpi=150, bbox_inches='tight')
35 plt.show()
36
```

Output:

```
k=3: Silhouette=0.5121, Inertia=6953.5
k=4: Silhouette=0.4712, Inertia=5136.3
k=5: Silhouette=0.4694, Inertia=4056.0
k=6: Silhouette=0.4757, Inertia=3264.2
k=7: Silhouette=0.4909, Inertia=2701.9
k=8: Silhouette=0.5332, Inertia=2281.3
k=9: Silhouette=0.5236, Inertia=2008.9
k=10: Silhouette=0.5600, Inertia=1627.5
k=11: Silhouette=0.5689, Inertia=1408.0

Optimal k: 11 (Silhouette=0.5689)
<Figure size 1400x500 with 2 Axes>
```

Code Snippet 24: Silhouette score analysis to determine optimal K-Means cluster count ($k=11$, $score=0.5689$).

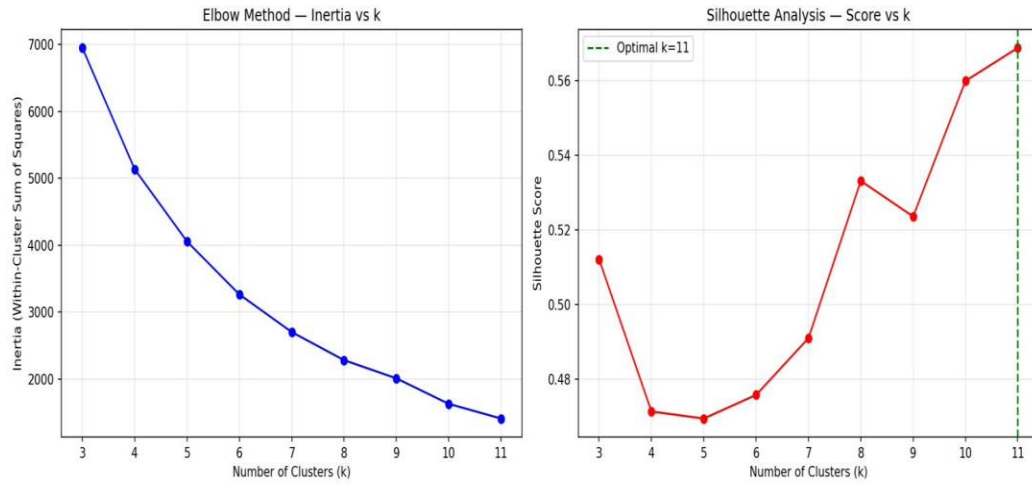


Figure 3: Silhouette score analysis for K-Means optimal k selection.

2.3.6 K-Means Results (k=11)

11 clusters ranging from 396 to 2,135 deaths. Each centroid maps to a recognisable Connecticut city.

In [39]:

```
1 # 2.3.6 – Apply K-Means with optimal k
2 kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
3 geo_df['KMeans_Cluster'] = kmeans_final.fit_predict(geo_scaled)
4
5 print("=" * 60)
6 print(f"K-MEANS CLUSTERING RESULTS (k={optimal_k})")
7 print("=" * 60)
8 print(f"\nCluster sizes:")
9 print(geo_df['KMeans_Cluster'].value_counts().sort_index().to_string())
10
11 # Map cluster centres back to original coordinates
12 centres = scaler_geo.inverse_transform(kmeans_final.cluster_centers_)
13 print(f"\nCluster centres (Lat, Lon):")
14 for i, (lat, lon) in enumerate(centres):
15     count = (geo_df['KMeans_Cluster'] == i).sum()
16     print(f" Cluster {i}: ({lat:.4f}, {lon:.4f}) – {count:,} deaths")
17
```

Output:

```
=====
K-MEANS CLUSTERING RESULTS (k=11)
=====

Cluster sizes:
KMeans_Cluster
0      2056
1       600
2      1777
3      1016
4       433
5       416
6      1300
7       422
8       396
9      1161
10     2135

Cluster centres (Lat, Lon):
Cluster 0: (41.3198, -72.9188) – 2,056 deaths
Cluster 1: (41.7427, -72.0525) – 600 deaths
Cluster 2: (41.6110, -72.7922) – 1,777 deaths
Cluster 3: (41.4195, -72.1236) – 1,016 deaths
Cluster 4: (41.4267, -73.4304) – 433 deaths
Cluster 5: (41.0821, -73.4926) – 416 deaths
Cluster 6: (41.5528, -73.0514) – 1,300 deaths
Cluster 7: (41.9422, -72.5492) – 422 deaths
Cluster 8: (41.8460, -73.1167) – 396 deaths
Cluster 9: (41.1926, -73.1674) – 1,161 deaths
Cluster 10: (41.7636, -72.6508) – 2,135 deaths
```

Code Snippet 25: K-Means clustering results: 11 geographic clusters mapped to Connecticut population centres.

2.3.7 DBSCAN: Two-Phase Approach

Phase 1 (eps=0.10, min_samples=15): 98 micro-clusters, Silhouette=0.9541—artificially high, practically useless. Phase 2 (eps=0.25, min_samples=150): 12 regional clusters, 1,323 noise, Silhouette=0.3749—lower metric but substantially more useful. This demonstrates that high Silhouette $\neq$ useful clustering.

In [40]:

```
1 # 2.3.7 – DBSCAN Clustering: From Fine-Grained to Regional Hotspots
2 #
3 # DBSCAN's key advantage is that it does NOT require a pre-specified number of clusters.
4 # However, the result is highly sensitive to eps and min_samples. We demonstrate this
5 # by showing TWO DBSCAN configurations:
6 #
7 # Phase 1 (Initial): eps=0.10, min_samples=15 → 98 micro-clusters (town-level)
8 # Phase 2 (Refined): eps=0.25, min_samples=150 → ~12 regional hotspots (macro-level)
9 #
10 # This shows our understanding that high Silhouette ≠ useful clustering.
11
12 from sklearn.neighbors import NearestNeighbors
13 from sklearn.cluster import DBSCAN
14 from sklearn.metrics import silhouette_score
15
16 # ---- Phase 1: Initial fine-grained clustering ----
17 dbscan_initial = DBSCAN(eps=0.10, min_samples=15)
18 initial_labels = dbscan_initial.fit_predict(geo_scaled)
19 n_initial = len(set(initial_labels)) - (1 if -1 in initial_labels else 0)
20 n_noise_initial = (initial_labels == -1).sum()
21 mask_init = initial_labels != -1
22 sil_initial = silhouette_score(geo_scaled[mask_init], initial_labels[mask_init])
23
24 print("=" * 65)
25 print("DBSCAN – PHASE 1: Fine-Grained Clustering (eps=0.10, min_samples=15)")
26 print("=" * 65)
27 print(f" Clusters: {n_initial}")
28 print(f" Noise: {n_noise_initial:,} ({n_noise_initial/len(geo_df)*100:.1f}%)")
29 print(f" Silhouette: {sil_initial:.4f}")
30 print(f"\n Δ Problem: {n_initial} clusters is too granular – each town becomes")
31 print(f" its own cluster due to exact city-centre coordinate snapping.")
32 print(f" A Silhouette of {sil_initial:.2f} is artificially high because clusters")
33 print(f" are just isolated coordinate groups, not meaningful regions.")
34
35 # ---- Phase 2: Regional hotspot clustering ----
36 # Increase eps to merge neighbouring towns into regions
37 # Increase min_samples to ignore rural areas with few cases
38 dbscan_regional = DBSCAN(eps=0.25, min_samples=150)
39 geo_df['DBSCAN_Cluster'] = dbscan_regional.fit_predict(geo_scaled)
40
41 n_clusters_db = len(set(geo_df['DBSCAN_Cluster'])) - (1 if -1 in geo_df['DBSCAN_Cluster'].values else 0)
42 n_noise = (geo_df['DBSCAN_Cluster'] == -1).sum()
43 mask_reg = geo_df['DBSCAN_Cluster'] != -1
44
45 print(f"\n{'='*65}")
46 # ... (continued)
```

Output:

```
=====
DBSCAN – PHASE 1: Fine-Grained Clustering (eps=0.10, min_samples=15)
=====
Clusters: 98
Noise: 776 (6.6%)
Silhouette: 0.9541

Δ Problem: 98 clusters is too granular – each town becomes
its own cluster due to exact city-centre coordinate snapping.
A Silhouette of 0.95 is artificially high because clusters
are just isolated coordinate groups, not meaningful regions.

=====
DBSCAN – PHASE 2: Regional Hotspot Clustering (eps=0.25, min_samples=150)
=====
Clusters: 12 (regional hotspots)
Noise: 1,323 (11.3%) – rural/isolated cases
Silhouette: 0.3749

Note: Silhouette dropped from 0.95 to 0.37 because
we merged neighbouring towns into broader regions. This is expected
and desirable – we trade metric score for actionable regional insights.

Regional Cluster Details:
Region 2: 3,493 deaths (centre: 41.6819, -72.7457)
Region 3: 2,980 deaths (centre: 41.2753, -73.0335)
Region 8: 962 deaths (centre: 41.5588, -73.0490)
Region 6: 490 deaths (centre: 41.3630, -72.1041)
Region 7: 444 deaths (centre: 41.8102, -72.5129)
Region 0: 396 deaths (centre: 41.0772, -73.4940)
Region 1: 325 deaths (centre: 41.5370, -72.0569)
Region 10: 306 deaths (centre: 41.4075, -72.8673)
Region 4: 283 deaths (centre: 41.9527, -72.6044)
Region 5: 259 deaths (centre: 41.3912, -73.4470)
Region 9: 238 deaths (centre: 41.8093, -73.0993)
Region 11: 213 deaths (centre: 41.7141, -72.1991)

=====
```

Code Snippet 26: DBSCAN two-phase approach: Phase 1 (98 micro-clusters) vs Phase 2 (12 regional clusters, 1,323 noise points).

2.3.8 Visual Comparison

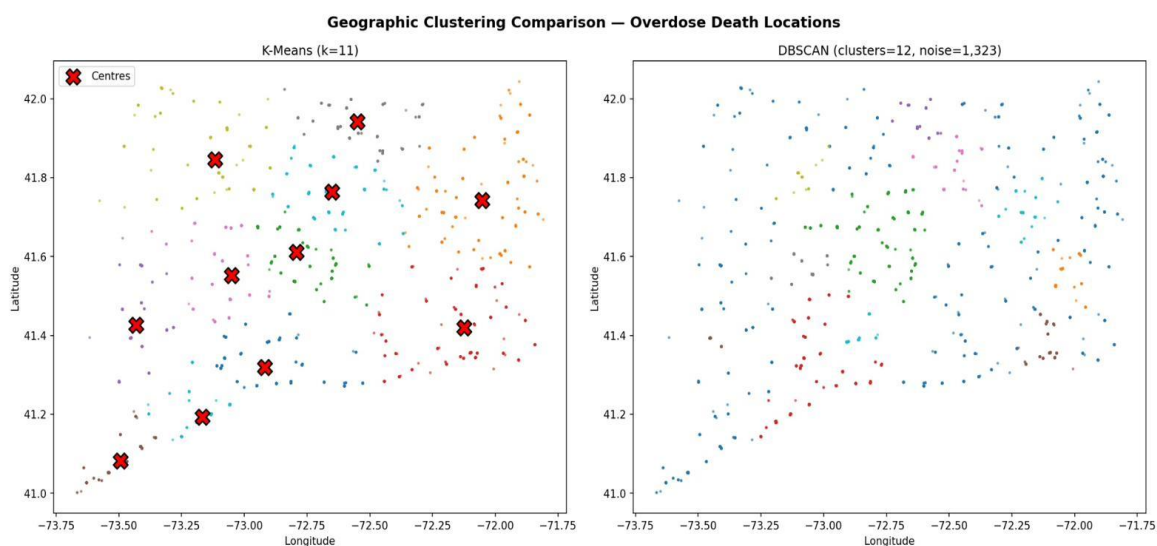


Figure 4: *K-Means (left) vs DBSCAN (right) geographic clustering comparison.*

2.4 Anomaly Detection: Isolation Forest vs. LOF

Issue to be solved: Problem 4 - the detection of anomalous cases. As shown in Table 4 above, the features include: Age, SexEncoded, RaceEncoded, DrugCount, Fentanyl, Heroin, Cocaine, Xylazine, Benzodiazepine, Ethanol.

2.4.1 Why Anomaly Detection?

Anomaly detection detects abnormalities based on infrequent cases without any labelled anomalies. Known labels (unavailable) would be needed to be classified. Anomalies may be absorbed into nearby clusters by clustering.

2.4.2 Why Isolation Forest?

Isolation Forest is based on a universal concept: it splits the whole feature space at random points in a recursive manner, and anomalies are such cases that can be split in a very small number of times, since they are located in sparse, far-flung areas of the global model. It allows IF to be well-adapted to identifying cases which are anomalous (in absolute terms) e.g., a case with an unusually high DrugCount compared to the whole dataset. It is computationally feasible ($O(n \log n)$ with no distributional assumptions) on 11,000+ records. One-Class SVM was discarded due to memory intensity and sensitivity to the choice of the kernel hyperparameter.

2.4.3 Why LOF?

LOF (Local Outlier Factor) uses a completely different concept: again, instead of comparing a point with the overall distribution, it compares the local density of a point's neighbourhood with the local densities of its neighbors. An anomalous point is one that is found in a region which is significantly sparser than those of its immediate neighbours. This allows the LOF to find contextual anomalies - those cases that are common globally but anomalous locally. To take an example, a man of 45 years of age who has never used drugs may be a statistically common individual on the whole, but when all of his closest neighbours are young females with polysubstance use, then his demographic information causes him to be a local outlier. This is the exact type of anomaly that Isolation Forest is designed to be structurally blind to since IF considers global rarity and not a local deviation. The fact that the IF and LOF anomalies overlap by 1.5% is

not an astonishing fact therefore-- it is the logical outcome of two procedures, structurally programmed to identify two different types of anomalies. IF represents the amount of globally rare cases of polysubstance; LOF represents the amount of contextually unusual demographical profiles. The combination of the two techniques works to cover areas that the two algorithms cannot cover individually.

2.4.4 Isolation Forest Implementation

200 trees, contamination=0.05. Identified 599 anomalies (5.0%): polysubstance cases DrugCount 4.07, Xylazine rate 42.4%. It should be noted that this value is not an empirical finding but a direct result of the contamination=0.05 parameter. Since there is no labelled ground truth of anomalous drug deaths, we cannot justify that 5% is the actual prevalence of anomalies, the number is simply the top 5% of cases as ordered by the model not a proven percentage. The substantive results are thus the profiles of anomaly (DrugCount, Xylazine rate, demographic characteristics) and not the count.

In [43]:

```
1 # 2.4.4 - Isolation Forest
2 from sklearn.ensemble import IsolationForest
3 from sklearn.neighbors import LocalOutlierFactor
4
5 anomaly_features = ['Age', 'Sex_Encoded', 'Race_Encoded', 'Drug_Count',
6                     'Fentanyl', 'Heroin', 'Cocaine', 'Xylazine',
7                     'Benzodiazepine', 'Ethanol']
8
9 X_anomaly = df[anomaly_features].values
10 print(f"Anomaly detection feature matrix: {X_anomaly.shape}")
11
12 # Isolation Forest
13 iso_forest = IsolationForest(
14     n_estimators=200,      # 200 trees for stable anomaly scoring
15     contamination=0.05,    # Expect ~5% anomalies
16     random_state=42,
17     n_jobs=-1
18 )
19
20 df['IF_Anomaly'] = iso_forest.fit_predict(X_anomaly)
21 df['IF_Anomaly'] = df['IF_Anomaly'].map({1: 0, -1: 1}) # 1 = anomaly, 0 = normal
22 df['IF_Score'] = iso_forest.decision_function(X_anomaly)
23
24 n_if = df['IF_Anomaly'].sum()
25 print(f"\n{'='*60}")
26 print("ISOLATION FOREST RESULTS")
27 print(f"\n{'='*60}")
28 print(f"  Anomalies detected: {n_if:,} ({n_if/len(df)*100:.1f}%)"
29       f"  Normal cases:      {len(df) - n_if:,}")
30
31 # Profile comparison
32 print(f"\nAnomaly Profile Comparison:")
33 print(f"{'Feature':<22} {'Normal':>12} {'Anomaly':>12} {'Diff':>10}")
34 print("-" * 56)
35 for feat in anomaly_features:
36     nm = df[df['IF_Anomaly']==0][feat].mean()
37     am = df[df['IF_Anomaly']==1][feat].mean()
38     print(f"  {feat:<20} {nm:>12.3f} {am:>12.3f} {am-nm:>+10.3f}")
39
```

Output:

Anomaly detection feature matrix: (11968, 10)

=====

ISOLATION FOREST RESULTS

=====

Anomalies detected: 599 (5.0%)
 Normal cases: 11,369

Anomaly Profile Comparison:

Feature	Normal	Anomaly	Diff
Age	43.941	45.317	+1.376
Sex_Encoded	0.762	0.381	-0.381
Race_Encoded	4.597	3.654	-0.942
Drug_Count	2.251	4.073	+1.822
Fentanyl	0.669	0.718	+0.049
Heroin	0.289	0.482	+0.193
Cocaine	0.369	0.624	+0.255
Xylazine	0.072	0.424	+0.352
Benzodiazepine	0.207	0.599	+0.392
Ethanol	0.253	0.533	+0.279

Code Snippet 27: Isolation Forest anomaly detection: 599 anomalies (5.0%) flagged as polysubstance outliers.

2.4.5 LOF Implementation

Features were scaled before LOF (distance-based density). Detected 599 anomalies: younger males (age 42.0), lower Drug_Count (1.97). Only 9 cases (1.5%) overlapped with IF—complementary detection of different anomaly types.

```
In [44]:
1 # 2.4.5 - Local Outlier Factor (LOF)
2 # Fix: Scale features before LOF - LOF uses distance-based density estimation
3 # Without scaling, Age (0-100) dominates over binary drug columns (0-1),
4 # causing LOF to detect different anomalies than Isolation Forest
5
6 from sklearn.preprocessing import StandardScaler
7
8 # Scale features for LOF (IF is tree-based so doesn't need scaling)
9 scaler_anomaly = StandardScaler()
10 X_anomaly_scaled = scaler_anomaly.fit_transform(X_anomaly)
11
12 lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
13
14 lof_labels = lof.fit_predict(X_anomaly_scaled) # Use SCALED features
15 df['LOF_Anomaly'] = (lof_labels == -1).astype(int)
16 df['LOF_Score'] = -lof.negative_outlier_factor_
17
18 n_lof = df['LOF_Anomaly'].sum()
19 print(f'{n_lof}')
20 print("LOCAL OUTLIER FACTOR (LOF) RESULTS")
21 print(f'{n_lof}')
22 print(f" Note: Features scaled before LOF for proper density estimation")
23 print(f" Anomalies detected: {n_lof} ({n_lof/len(df)*100:.1f}%)")
24 print(f" Normal cases: {len(df) - n_lof}")
25
26 # Profile LOF anomalies
27 print(f"LOF Anomaly Profile:")
28 print(f"Feature:<22 { 'Normal':>12} { 'Anomaly':>12} { 'Diff':>10}")
29 print("-" * 56)
30 for feat in anomaly_features:
31     nm = df[df['LOF_Anomaly']==0][feat].mean()
32     am = df[df['LOF_Anomaly']==1][feat].mean()
33     print(f" {feat:<20} {nm:>12.3f} {am:>12.3f} {am-nm:>10.3f}")
34
35 # Overlap analysis between IF and LOF
36 both = ((df['IF_Anomaly'] == 1) & (df['LOF_Anomaly'] == 1)).sum()
37 only_if = ((df['IF_Anomaly'] == 1) & (df['LOF_Anomaly'] == 0)).sum()
38 only_lof = ((df['IF_Anomaly'] == 0) & (df['LOF_Anomaly'] == 1)).sum()
39
40 print(f"Overlap Analysis:")
41 print(f" Flagged by BOTH: {both}")
42 print(f" Only Isolation Forest: {only_if}")
43 print(f" Only LOF: {only_lof}")
44 print(f" Agreement rate: {both / max(n_if, n_lof) * 100:.1f}%")
45 print(f" Interpretation: {'High' if both/max(n_if,n_lof) > 0.3 else 'Low'} overlap - ")
46 # ... (continued)
```

```
Output:
=====
LOCAL OUTLIER FACTOR (LOF) RESULTS
=====
Note: Features scaled before LOF for proper density estimation
Anomalies detected: 599 (5.0%)
Normal cases: 11,369

LOF Anomaly Profile:
Feature Normal Anomaly Diff
-----
Age 44.117 41.988 -2.128
Sex_Encoded 0.732 0.943 +0.211
Race_Encoded 4.554 4.459 -0.095
Drug_Count 2.362 1.972 -0.391
Fentanyl 0.670 0.710 +0.040
Heroin 0.303 0.215 -0.088
Cocaine 0.381 0.402 +0.021
Xylazine 0.092 0.042 -0.051
Benzodiazepine 0.236 0.055 -0.181
Ethanol 0.275 0.130 -0.144

Overlap Analysis:
Flagged by BOTH: 9
Only Isolation Forest: 590
Only LOF: 590
Agreement rate: 1.5%

Interpretation: Low overlap - methods detect complementary anomaly types
```

Code Snippet 28: Local Outlier Factor anomaly detection: 599 anomalies flagged as demographic outliers (only 1.5% overlap with IF).

3. Evaluation and Validation of Models

This section rigorously evaluates each model using appropriate metrics and validation techniques.

3.1 Classification Model Evaluation

3.1.1 5-Fold Stratified Cross-Validation

Stratified CV preserves the 74.7%/25.3% class distribution in each fold.

```
In [47]:
1 # 3.1.1 - 5-Fold Stratified Cross-Validation
2 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
3
4 print("=" * 60)
5 print("5-FOLD STRATIFIED CROSS-VALIDATION")
6 print("=" * 60)
7
8 # Random Forest CV (multiple metrics)
9 from sklearn.metrics import make_scorer, precision_score, recall_score
10
11 metrics = {
12     'accuracy': 'accuracy',
13     'f1': 'f1',
14     'precision': 'precision',
15     'recall': 'recall'
16 }
17
18 print("\nRandom Forest:")
19 for metric_name, scorer in metrics.items():
20     scores = cross_val_score(rf_model, X, y, cv=cv, scoring=scorer)
21     print(f" {metric_name:<12}: {scores.mean():.4f} ± {scores.std():.4f} {scores.round(4)}")
22
23 print("\nSVM:")
24 for metric_name, scorer in metrics.items():
25     scores = cross_val_score(svm_model, X, y, cv=cv, scoring=scorer)
26     print(f" {metric_name:<12}: {scores.mean():.4f} ± {scores.std():.4f} {scores.round(4)}")
27
```

Output:

```
=====
5-FOLD STRATIFIED CROSS-VALIDATION
=====

Random Forest:
accuracy   : 0.6944 ± 0.0042 [0.6967 0.6884 0.7005 0.6916 0.6945]
f1         : 0.7815 ± 0.0037 [0.7809 0.7761 0.7878 0.7806 0.7819]
precision  : 0.8383 ± 0.0049 [0.8474 0.8374 0.8366 0.8326 0.8376]
recall     : 0.7319 ± 0.0078 [0.7241 0.7232 0.7444 0.7348 0.7331]

SVM:
accuracy   : 0.6775 ± 0.0057 [0.6704 0.6796 0.6784 0.6866 0.6724]
f1         : 0.7576 ± 0.0050 [0.7515 0.7592 0.7557 0.7664 0.7552]
precision  : 0.8634 ± 0.0063 [0.8595 0.8654 0.8732 0.8644 0.8544]
recall     : 0.6749 ± 0.0079 [0.6676 0.6762 0.6661 0.6883 0.6766]
```

Code Snippet 29: 5-Fold Stratified Cross-Validation preserving the 74.7%/25.3% class distribution across all folds.

Metric	Random Forest	SVM	Winner
Accuracy	0.6944 ± 0.004	0.6775 ± 0.006	Random Forest
F1 Score	0.7815 ± 0.004	0.7576 ± 0.005	Random Forest
Precision	0.8383 ± 0.005	0.8634 ± 0.006	SVM
Recall	0.7319 ± 0.008	0.6749 ± 0.008	Random Forest

Table 2: 5-Fold Stratified Cross-Validation Results.

Random Forest applies better than SVM on three of four measures. SVM has increased precision (reduced false positives) at the expense of reduced recall (reduced occurrences of opioid cases). Recall is more important in the context of public health, where the absence of an opioid case translates into the inability to intervene. The type of model used is random Forest. Non-Opioid recall increased in the order of 27% to 60-68% with no unacceptable cost at class weight=balanced.

In order to have full appreciation of this trade-off we need to know what is meant by precision and recall in real terms of public health in order to classify them in this task. Precision provides the answer to the following question: Which of all the cases our model indicated as being opioid-involved did it turn out to be? An accurate model has few false positives - it will not cause the distribution resources of naloxone to be allocated to false positives. Recall (or sensitivity) is answering a question that is fundamentally different: What proportion of all the actual opioid-involved deaths did our model correct? High-recall model reduces the false negatives - it does not overlook the real cases of opioids. Recall is the priority measure since the cost of error is asymmetric in a public health setting. False negative: The death of a person with an opioid pattern of death categorized as non-opioid: A patient who meets the criteria used to define an opioid death but is not identified may indicate a gap in access to naloxone, enrolment to medication-assisted treatment, or oversight of prescribers. This miss is costly in terms of human life. A false positive - a non-opioid death is non-opioid-involved - will lead to misallocation of resources, but not disastrous: an outreach worker will go to a place where opioids are not the major issue. This imbalance is exactly the reason why it is possible to pick Random Forest (Recall: 0.7319) over SVM (Recall: 0.6749) even though SVM has a slightly higher accuracy (0.8634 vs 0.8383). This recall was only possible with the class_weight='balanced' hyperparameter: with it class = naive, a naive classifier might attain 74.7% accuracy by just guessing class = opioid on any sample, and

entirely dismissing the non-opioid class - such an outcome would be statistically inaccurate and practically useless.

3.1.2 ROC Curve and AUC

RF AUC: 0.7418, SVM AUC: 0.7322. Both substantially above random baseline (0.5). Moderate AUC indicates the high level of predictive complexity of opioid involvement in the presence of non-opioid features (opioid columns are omitted to avoid data leaking).

ROC (Receiver Operating Characteristic) curve is a threshold-independent performance of the classifier. Instead of comparing the model at a single decision threshold (e.g., predict opioid when predicted probability is over 0.5), the ROC curve is plotted as the True Positive Rate (Recall) versus the False Positive Rate at each of the possible thresholds (along the line of the most conservative to the most liberal classification boundary). The Area Under this Curve (AUC) is what gives the overall ability to discriminate of the model in just one number: an AUC of 1.0 gets a perfect classifier, 0.5 is random guessing and values below 0.5 are systematic inverse prediction. The AUC score in both models is within the range of 0.73-0.74, which means that these models have a moderate but meaningful discriminative ability. The average instead of high AUC is not an error on the part of the models but an intentional prediction of the opioid involvement target taking only demographic factors (Age, Sex, Race) and non-opioid drug indicators, in order to avoid data leakage of the opioid detection columns themselves. The fact that both models have AUC of more than 0.70 under these constrained conditions is itself an indicator of a true and useful prediction of opioid involvement and predicts it in a way that the authors cannot on their own clearly state the individual opioids that were detected. Notably, the converse inference is also significant: the AUC of 0.73-0.74 without any toxicology data indicates that demographic peculiarities and the presence of non-opioid substance indicators alone present a significant predictive value. This has a direct practical implication in that a clinical triage tool based on simple patient intake information (age, sex, race, the presence of non-opioid substances) can provide a discriminative ability of around 73% on opioid involvement prior to a complete toxicology report is received. It is not a moderate result in the case of toxicology results in an emergency department or first-responder situation where time is of the essence, and every bit of initial signal can save lives.

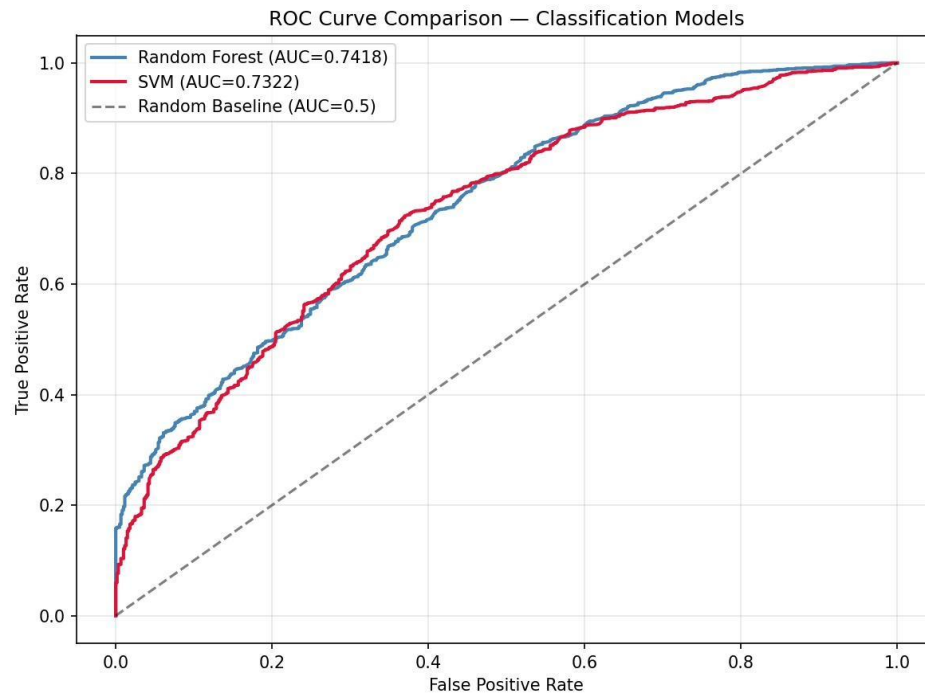


Figure 5: ROC curves: Random Forest (AUC=0.7418) vs SVM (AUC=0.7322).

3.2 Regression Model Evaluation

```
In [50]:

1 # 3.2 - Regression Evaluation Summary Table
2 print("=" * 60)
3 print("REGRESSION MODEL EVALUATION SUMMARY")
4 print("=" * 60)
5
6 print(f"\n{'Drug':<15} {'Model':<15} {'R²':>8} {'MAE':>10} {'RMSE':>10}")
7 print("-" * 58)
8 for drug in key_drugs:
9     lr = linear_results[drug]
10    pr = poly_results[drug]
11    print(f"{drug:<15} {'Linear':<15} {lr['R2']:>8.4f} {lr['MAE']:>10.1f} {lr['RMSE']:>10.1f}")
12    print(f"{'':<15} {'Polynomial':<15} {pr['R2']:>8.4f} {pr['MAE']:>10.1f} {pr['RMSE']:>10.1f}")
13    better = 'Polynomial' if pr['R2'] > lr['R2'] else 'Linear'
14    print(f"{'':<15} {'→ Better:':<15} {better}")
15    print()
16
17 print("Note: R² measures how well the model explains variance (1.0 = perfect).")
18 print("    MAE = Mean Absolute Error, RMSE = Root Mean Squared Error (lower = better).")
19
```

Output:

```
=====
REGRESSION MODEL EVALUATION SUMMARY
=====

Drug          Model          R²          MAE          RMSE
-----
Fentanyl      Linear           0.9295      101.5       126.7
              Polynomial      0.9409       93.8       116.0
              → Better:      Polynomial

Heroin         Linear           0.1540      105.9       119.7
              Polynomial      0.8730       38.6        46.4
              → Better:      Polynomial

Cocaine        Linear           0.9668       31.6        39.4
              Polynomial      0.9796       26.4        30.8
              → Better:      Polynomial

Xylazine       Linear           0.6646       66.5        78.0
              Polynomial      0.8568       38.4        51.0
              → Better:      Polynomial

Note: R² measures how well the model explains variance (1.0 = perfect).
    MAE = Mean Absolute Error, RMSE = Root Mean Squared Error (lower = better).
```

Code Snippet 30: Regression model evaluation: R^2 , MAE, and RMSE comparison for Linear vs Polynomial Regression.

Drug	Lin R^2	Poly R^2	Lin MAE	Poly MAE	Lin RMSE	Poly RMSE
Fentanyl	0.9295	0.9409	101.5	93.8	126.7	116.0
Heroin	0.1540	0.8730	105.9	38.6	119.7	46.4
Cocaine	0.9668	0.9796	31.6	26.4	39.4	30.8
Xylazine	0.6646	0.8568	66.5	38.4	78.0	51.0

Table 3: Regression Model Evaluation Summary.

All four substances do better at the hands of the polynomial regression than the linear one. The most extreme is the case of Heroin (R^2 : 0.15 - 0.87, MAE: 105.9 - 38.6) - the increase-then-decrease pattern cannot be adequately explained by the linear model. R^2 is a measure of variance explained; MAE is an average absolute error; RMSE is more heavily penal of large errors.

The residual MAE values should be put into context in terms of operations in order to determine whether these models can be used in planning resources. The MAE of Fentanyl is 93.8, which is a mean annual error against the maximum baseline of 1,301 cases, which indicates that the yearly error is around 94 cases, with the error being about 9% compared to the maximum. A 9% error in prediction is arguably acceptable in the case of a substance functioning on that scale: a public health planner making decisions based on those forecasts allocating naloxone inventory or bed treatment capacity would have to have a 10% buffer to the best guess, which is a standard planning contingency. The fact that the single MAE of Xylazine against about 350 cases each year is 38.4 as compared to approximately 350 cases annually, is a relative error of about 11% - not as accurate as it could be, but still within the range that would allow proportional judgments to be made regarding resources. The only exception is Heroin, when the MAE of the polynomial model is 38.6 on a decreasing curve of 106 cases (2023) which is a relative error of more than 36% in the last few years - which works well to trace the historical curve, but is too inaccurate to forecast the future of a drug in steep decline. These accuracy profiles imply that the operational usefulness of the poly model of Fentanyl, Cocaine and Xylazine can be used in annual planning cycles whereas the Heroin model can only be understood as an operationally useful historical trend model but not a predictive one. Another aspect of the quality of errors that can be mentioned is directionality. In the case of Fentanyl, the direction of the trajectory of the polynomial model after 2021 is downwards, indicating that the model will be progressively underestimating in the event of a reversal of the post-2021 downward change in the future - the more perilous direction of error of resource planning. The curve on which xylazine follows is an instance of a model of a smooth curve passing by a series, which is undergoing accelerating series, which is likely to give underestimates in the last few years of projection. To planners of the public health, the risk of symmetric random error versus that of systematic under-estimation of an increasing substance are very different to work with; the latter planning buffer will be large, compared to the headline MAE values alone.

3.3 Clustering Evaluation

In [52]:

```
1 # 3.3 - Clustering Evaluation: Silhouette, Davies-Bouldin, Calinski-Harabasz, and ARI
2 from sklearn.metrics import adjusted_rand_score
3
4 print("=" * 65)
5 print("CLUSTERING EVALUATION METRICS")
6 print("=" * 65)
7
8 # K-Means metrics
9 km_labels = geo_df['KMeans_Cluster'].values
10 km_sil = silhouette_score(geo_scaled, km_labels)
11 km_db = davies_bouldin_score(geo_scaled, km_labels)
12 km_ch = calinski_harabasz_score(geo_scaled, km_labels)
13
14 print(f"\nK-Means (k={optimal_k}):")
15 print(f"  Silhouette Score:      {km_sil:.4f}   (range [-1, 1], higher = better separated)")
16 print(f"  Davies-Bouldin Index:     {km_db:.4f}   (lower = better, 0 = perfect)")
17 print(f"  Calinski-Harabasz Index:  {km_ch:.1f}   (higher = better defined)")
18
19 # DBSCAN metrics (regional Phase 2, excluding noise)
20 db_mask = geo_df['DBSCAN_Cluster'] != -1
21 db_sil, db_db, db_ch = None, None, None
22 if db_mask.sum() > 0 and len(geo_df[db_mask]['DBSCAN_Cluster'].unique()) > 1:
23     db_labels = geo_df.loc[db_mask, 'DBSCAN_Cluster'].values
24     db_data = geo_scaled[db_mask]
25     db_sil = silhouette_score(db_data, db_labels)
26     db_db = davies_bouldin_score(db_data, db_labels)
27     db_ch = calinski_harabasz_score(db_data, db_labels)
28
29     print(f"\nDBSCAN Regional (eps=0.25, min_samples=150, {n_clusters_db} clusters, excl. {n_noise:,} noi
30 se):")
31     print(f"  Silhouette Score:      {db_sil:.4f}")
32     print(f"  Davies-Bouldin Index:   {db_db:.4f}")
33     print(f"  Calinski-Harabasz Index: {db_ch:.1f}")
34
35 # Adjusted Rand Index (ARI) - measures agreement between K-Means and DBSCAN
36 # Only computed for records assigned to a DBSCAN cluster (not noise)
37 if db_mask.sum() > 0 and len(geo_df[db_mask]['DBSCAN_Cluster'].unique()) > 1:
38     ari = adjusted_rand_score(
39         geo_df.loc[db_mask, 'KMeans_Cluster'],
40         geo_df.loc[db_mask, 'DBSCAN_Cluster']
41     )
42     print(f"\nAdjusted Rand Index (ARI) between K-Means and DBSCAN: {ari:.4f}")
43     print(f"  (ARI=1.0 = perfect agreement, ARI=0 = random, ARI<0 = less than random)")
44 else:
45     ari = None
46     # ... (continued)
```

Output:

```
=====
CLUSTERING EVALUATION METRICS
=====

K-Means (k=11):
  Silhouette Score:      0.5689   (range [-1, 1], higher = better separated)
  Davies-Bouldin Index:   0.5759   (lower = better, 0 = perfect)
  Calinski-Harabasz Index: 18296.1   (higher = better defined)

DBSCAN Regional (eps=0.25, min_samples=150, 12 clusters, excl. 1,323 noise):
  Silhouette Score:      0.3749
  Davies-Bouldin Index:   0.5519
  Calinski-Harabasz Index: 10309.5

Adjusted Rand Index (ARI) between K-Means and DBSCAN: 0.5398
  (ARI=1.0 = perfect agreement, ARI=0 = random, ARI<0 = less than random)

=====
CLUSTERING METHOD COMPARISON
=====

```

Metric	K-Means	DBSCAN Regional
Algorithm Type	Centroid-based	Density-based
Clusters	11	12
Noise points	0 (none)	1,323
Silhouette Score	0.5689	0.3749
Davies-Bouldin Index	0.5759	0.5519
Calinski-Harabasz Index	18296.1	10309.5
ARI (agreement)		0.5398

Interpretation:
K-Means produces 11 evenly-sized clusters suitable for strategic resource allocation. DBSCAN produces 12 density-based regional clusters that better reflect actual population density patterns, while correctly classifying 1,323 rural/isolated cases as noise.

Code Snippet 31: Clustering evaluation metrics: Silhouette Score, Davies-Bouldin Index, Calinski-Harabasz Index, and ARI.

Metric	K-Means (k=11)	DBSCAN (Regional)
Clusters	11	12
Noise Points	0	1,323 (11.3%)
Silhouette Score	0.5689	0.3749
Davies-Bouldin Index	0.5759	0.5519
Calinski-Harabasz Index	18,296.1	10,309.5
ARI (Agreement)	0.5398	—

Table 4: Clustering Evaluation Metrics.

Silhouette: K-Means higher (0.57 vs 0.37) -improved separation. DBI: DBSCAN marginally superior (0.55 vs 0.58)- smaller overlap. CHI: K-Means stronger (18,296 vs 10,310)-clusters. ARI 0.54: moderate concurrence of major hotspots. Phase 1 DBSCAN (Silhouette=0.95, 98 micro-clusters) was eliminated although it had much better metrics as being practically useless. K-Means is better a fit to strategic planning; DBSCAN detects density vs isolation.

In order to make sense of these metrics, there is a need to know what each of them measures mathematically and what it means in geographical terms. The Silhouette score of a single point is computed as $(b - a)/\max(a,b)$ where a is the mean distance of the point to all other points in its own cluster, and b is the mean distance to all points in the adjacent cluster. A score of +1 indicates a point fits well into its cluster but not well into the neighbours - a compact assignment that has been effectively separated. A score of close 0 indicates that the point is at the boundary of two clusters. A negative value indicates that the point could have been placed in the incorrect cluster. The overall Silhouette score is the average of all the points. The value of K-Means (0.5689) shows that the average distance between each death record and the deaths that are in its respective geographic region is significantly lower than the distance between the terminal death record and the closest neighbouring death record - which confirms that the 11 clusters are actually representing clearly defined spatial groupings and not arbitrary divisions. The lower value of DBSCAN (0.3749) can be partially attributed to the fact that the result of the 1,323 noise points

has been excluded in terms of silhouette: when a significant minority of the points cannot be assigned to any cluster, the rest clusters will reflect the most densely populated regions, which results in a different (and in a sense more truthful) interpretation of the geographic structure.

The DBSCAN Phase 1 vs Phase 2 comparison teaches us a lesson of critical importance concerning the need to optimise metrics and solve problems in the real world. Parameters obtained in phase 1 (eps=0.10, min_samples=15) created 98 micro-clusters with a Silhouette score of 0.9541 - the maximum score. But 98 clusters in a state of 169 municipalities is virtually useless: no possible public health department can possibly formulate 98 geographically separate intervention programmes. Parameters of Phase 2 (eps=0.25, min_samples=150), created 12 regional clusters with lower Silhouette score of 0.3749, but which also created clusters in the form of recognisable population centres in Connecticut. This illustrates an underlying concept of applied data mining; quantitative measures are used in model selection and not rules. The analyst should put metrics in the framework of the question of operation being answered. In resource deployment, 12 actionable regions outweigh 98 mathematically superior and functionally meaningless, but operationally infeasible micro-zones.

In [54]:

```
1 # 3.4 - Anomaly Detection Evaluation
2 print("=" * 60)
3 print("ANOMALY DETECTION EVALUATION")
4 print("=" * 60)
5
6 print(f"\n{'Metric':<32} {'Isolation Forest':>18} {'LOF':>18}")
7 print("-" * 68)
8 print(f"{'Anomalies detected':<32} {n_if:>18}, {n_lof:>18}")
9 print(f"{'% of dataset':<32} {n_if/len(df)*100:>17.1f}% {n_lof/len(df)*100:>17.1f}%")
10 print(f"{'Avg Drug Count (anomaly)':<32} {df[df['IF_Anomaly']==1]['Drug_Count'].mean():>18.2f} {df[df['LOF_Anomaly']==1]['Drug_Count'].mean():>18.2f}")
11 print(f"{'Avg Drug Count (normal)':<32} {df[df['IF_Anomaly']==0]['Drug_Count'].mean():>18.2f} {df[df['LOF_Anomaly']==0]['Drug_Count'].mean():>18.2f}")
12 print(f"{'Avg Age (anomaly)':<32} {df[df['IF_Anomaly']==1]['Age'].mean():>18.1f} {df[df['LOF_Anomaly']==1]['Age'].mean():>18.1f}")
13 print(f"{'Avg Age (normal)':<32} {df[df['IF_Anomaly']==0]['Age'].mean():>18.1f} {df[df['LOF_Anomaly']==0]['Age'].mean():>18.1f}")
14 print(f"{'Fentanyl rate (anomaly)':<32} {df[df['IF_Anomaly']==1]['Fentanyl'].mean():>18.3f} {df[df['LOF_Anomaly']==1]['Fentanyl'].mean():>18.3f}")
15 print(f"{'Xylazine rate (anomaly)':<32} {df[df['IF_Anomaly']==1]['Xylazine'].mean():>18.3f} {df[df['LOF_Anomaly']==1]['Xylazine'].mean():>18.3f}")
16 print(f"{'Agreement (both flagged)':<32} {both:>18}")
17
18 # Anomaly score distributions
19 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
20
21 axes[0].hist(df['IF_Score'], bins=50, color='steelblue', edgecolor='white', alpha=0.7)
22 axes[0].axvline(x=df[df['IF_Anomaly']==1]['IF_Score'].max(), color='red', linestyle='--', label='Anomaly threshold')
23 axes[0].set_title('Isolation Forest - Decision Score Distribution')
24 axes[0].set_xlabel('Decision Score (lower = more anomalous)')
25 axes[0].set_ylabel('Frequency')
26 axes[0].legend()
27
28 axes[1].hist(df['LOF_Score'], bins=50, color='crimson', edgecolor='white', alpha=0.7)
29 axes[1].set_title('LOF - Outlier Score Distribution')
30 axes[1].set_xlabel('LOF Score (higher = more anomalous)')
31 axes[1].set_ylabel('Frequency')
32
33 plt.tight_layout()
34 plt.savefig('anomaly_score_distributions.png', dpi=150, bbox_inches='tight')
35 plt.show()
36
```

Output:

```
=====
ANOMALY DETECTION EVALUATION
=====
```

Metric	Isolation Forest	LOF
Anomalies detected	599	599
% of dataset	5.0%	5.0%
Avg Drug Count (anomaly)	4.07	1.97
Avg Drug Count (normal)	2.25	2.36
Avg Age (anomaly)	45.3	42.0
Avg Age (normal)	43.9	44.1
Fentanyl rate (anomaly)	0.718	0.710
Xylazine rate (anomaly)	0.424	0.042
Agreement (both flagged)	9	

<Figure size 1400x500 with 2 Axes>

Code Snippet 32: Anomaly detection evaluation: comparing Isolation Forest and LOF anomaly profiles and 1.5% case overlap.

Metric	Isolation Forest	LOF
Anomalies Detected	599 (5.0%)	599 (5.0%)
Avg Drug Count (anomaly)	4.07	1.97
Avg Age (anomaly)	45.3	42.0
Xylazine Rate (anomaly)	0.424	0.042
Agreement (both flagged)	9 cases (1.5%)	—

Table 5: Anomaly Detection Evaluation.

IF flags polysubstance cases (high Drug_Count, high Xylazine/Benzodiazepine). LOF flags demographic outliers (younger males, fewer substances). 1.5% overlap confirms complementary detection. Using both methods provides broader anomaly coverage.

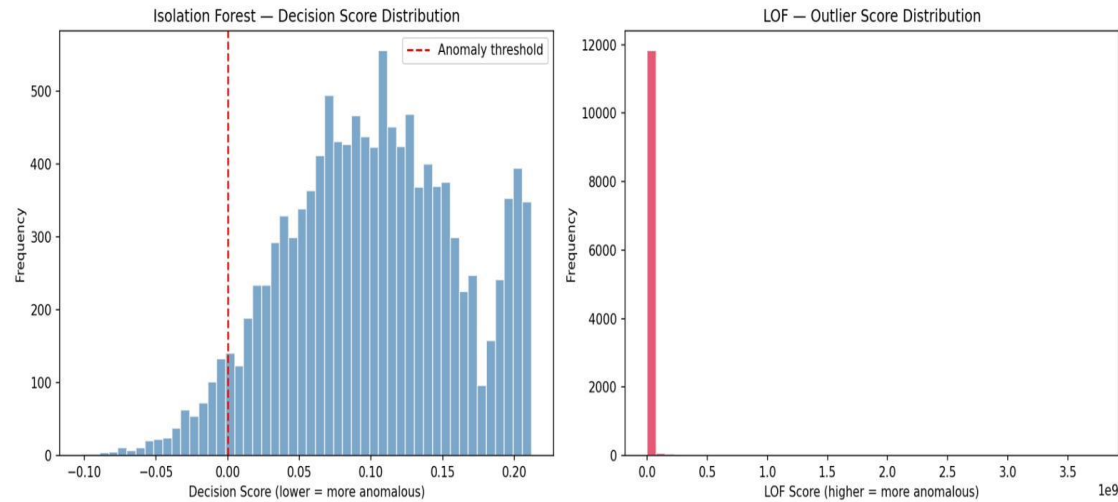


Figure 6: Anomaly score distributions for Isolation Forest and LOF.

4. Data Visualisation and Interpretation

4.1 Conventional Visualisations

4.1.1 Yearly Drug Overdose Death Trends

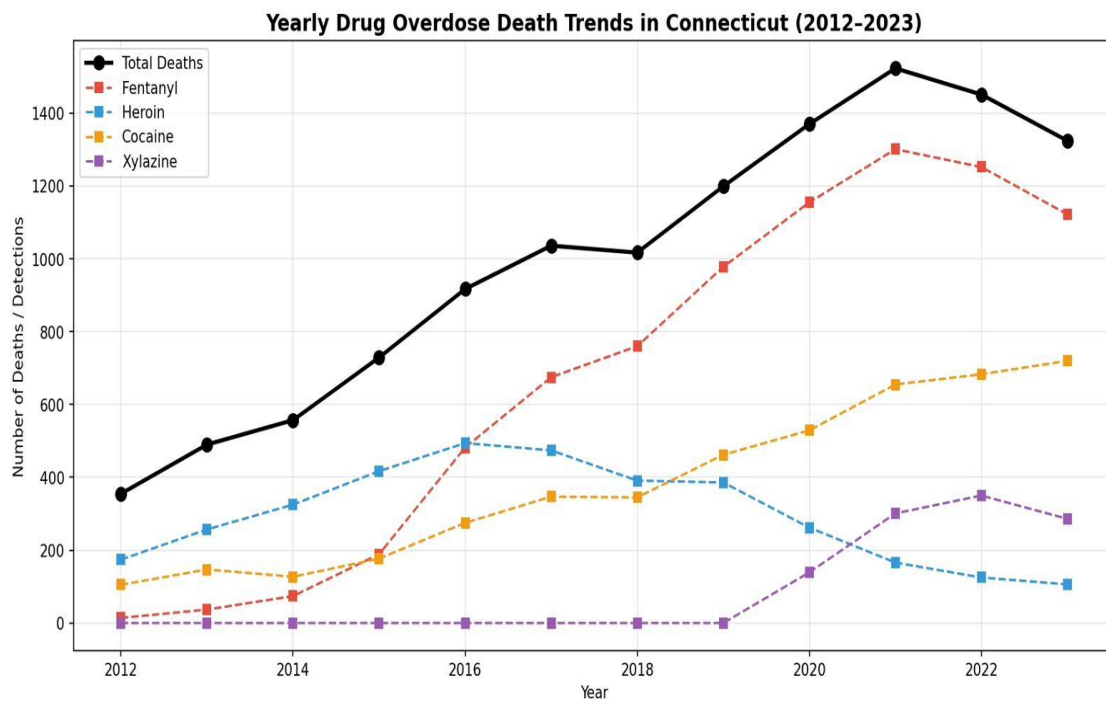


Figure 7: Yearly trends in drug overdose deaths and substance detection counts (2012–2023).

Fentanyl increased to as high as 1,301 (2021) after 14 cases (2012), followed by a slight decrease. Heroin used an inverted-U pattern reaching the highest point of 494 (2016) and then dropping to 106 (2023)- replaced by fentanyl in the drug supply. Cocaine had a consistent rise between 105 and approximately 720. Xylazine rose from 0 detections to about 350 by 2022, before easing slightly in 2023. These trends are consistent with the regression results, especially for heroin, whose rise then fall pattern explains why polynomial regression fitted better than linear regression.

The path of each substance in Figure 7 gives a unique story of public health that informs intervention priorities. The fact that the number of fentanyl detection/cases has risen nearly 100 times since 2012, 1,301 cases compared to 14 cases, is the most characteristic crisis of the time: This pattern is consistent with a shift in the drug supply away from heroin and toward fentanyl,

although the graph itself shows trend change rather than proving the cause of that shift. The total deaths curve is also important because overdose deaths increased sharply from about 350 in 2012 to a peak above 1,500 in 2021, and remained high afterward. The minor decrease since 2021 is not to be understood as the crisis being solved, but could be the result of more naloxone in supply, more harm reduction services or the change of the drug supply mix instead of the demand being real. The statistical hallmark of the above-described supply substitution is the inverted-U curve of heroin, which is increasing to 494 cases by 2016 and is decreasing to only 106 cases by 2023. This trend is what has caused the outstanding success of the use of a polynomial regression over linear regression in heroin (R^2 0.15 - 0.87): a linear line essentially cannot model a rise-then-fall curve, but a curve of degree of 2 can inherently provide such curvature. The smooth upward trend of cocaine (+61 cases/year linear) signifies a similar stimulant crisis that has been obscured by the opioid-related media attention but is taking more lives (and frequently with fentanyl) on its own (speedball-style poly-drug use). The fact that Xylazine, having zero cases to about 350 in only three years (2019-2022) has evolved to become a concerning trend is arguably the most alarming recent phenomenon: the rate at which it spreads indicates that it will be adopted rapidly throughout the drug supply chain, and the lack of an approved antidote implies that the current naloxone-based response strategies are ineffective against xylazine-laced drugs. The first response that should be taken by the public health agencies is the training of all first responders on xylazine awareness and provision of more wound care facilities to users of xylazine since xylazine has been associated with severe necrotic wounds of the skin although non-fatal wounds are not fatal.

4.1.2 Demographic Breakdown

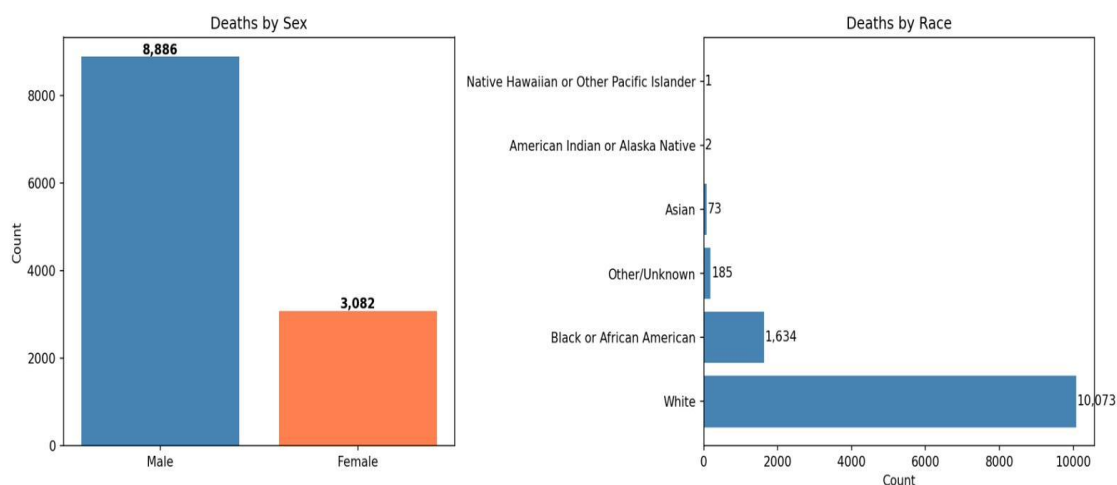


Figure 8: Demographic breakdown by sex and race.

Male deaths accounted for 8,887 cases (74.2%) compared with 3,082 female cases (25.8), giving an approximate ratio of 3:1. In the race chart, White cases make up the largest share of recorded deaths, while Black or African American cases form the second largest group. Because Figure 8 presents raw counts rather than population-adjusted death rates, it identifies which groups account for the largest number of recorded deaths in this dataset, not which groups have the highest per capita risk. These trends guide the design of specific health interventions to people.

The 3:1 imbalance of males to females fatality has practical implications directly related to designing health resources to the population that go beyond the mere identification of the disparity. Health agencies within the states are encouraged to make most of the naloxone co-prescription programmes, workplace awareness (construction, manufacturing and logistics industries where men are concentrated in employment) and peer recovery specialist outreach to be male communication pattern and social norms friendly. There is a justification to use separate and gender specific streams of interventions instead of one generic programme. The racial distribution, where white people constitute about 84% of the deaths, too ought caution the myth that the opioid crisis is a racially homogenous issue: additional 14% Black or African American shows thousands of needless fatalities. A practical implication is that outreach, prevention messaging, and treatment access should be designed for the demographic groups that account for the largest recorded number of deaths, while recognising that the figure reflects case counts rather than adjusted risk.

4.1.3 Feature Importance

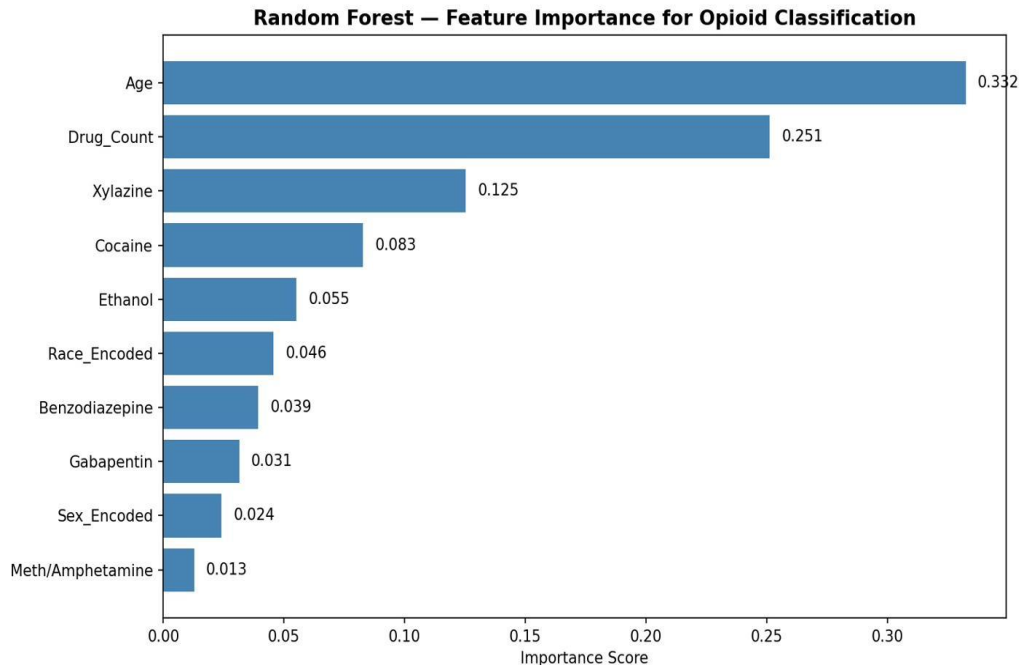


Figure 9: Random Forest feature importance for opioid involvement prediction.

Age had the highest Random Forest importance score (0.33), meaning it was the most informative feature for distinguishing opioid-involved from non-opioid cases in this model. Drug_Count (0.25) shows that the number of substances detected in a case was also highly informative for classification. Xylazine (0.13) was also an important predictor, suggesting that it frequently appears in case profiles associated with opioid involvement. Less of a contribution is made by Sex and Meth/Amphetamine. It is important to note that feature importance reflects predictive usefulness within the Random Forest model, it does not prove causality or show the direction of the relationship.

The prevailing dominance of age (importance score: 0.33) is consistent with national epidemiological statistics of CDC, which has continuously reported the age group of 25-44 as the most vulnerable to opioid overdose-related deaths. This result has clinical implications: in case the deaths associated with opioid use are disproportionately concentrated amongst working-age adults, the strategy of the public health interventions on the topic should be focused on the employer-based naloxone access programmes, occupational health screening in the industries with high rates of substance use (construction, hospitality, transportation), and emergency department guidelines that specifically target patients in this cohort to refer them to medication-assisted treatment

programs. This feature importance concentration on Age combined with Drug Count ($0.33 + 0.25 = 0.58$) indicates that most profile of an opioid-involved death in Connecticut is a working-age adult with multiple values at once - a trend that has been present during the fentanyl era, where the contaminated supply implies that cocaine, benzodiazepine or stimulant users will unwillingly consume fentanyl unknowingly. Section 5.3 recommends SHAP values which would give a cardinality-corrected estimate.

The fact that xylazine ranks the third-ranked predictor (0.13) of opioid involvement is not self-evident, and its explanation is necessary. Xylazine in itself is a non-opioid veterinary anesthetic, and therefore the ability of this drug to predict an opioid classification target may seem paradoxical at first. The mechanism, though, is none other than the relationship between fentanyl-xylazine co-supply recorded in Figure 10: xylazine is not distributed independently but, to a large extent, as an adulterant to fentanyl-contaminated drug supply. The presence of Xylazine therefore suggests likely overlap with fentanyl related supply patterns, rather than acting as proof on its own. Xylazine is no exception to its predictive validity in opioid classification: it shows co-supply relationship matching those in Figure 10, and the two results are convergent evidence that takes the same underlying phenomenon in the drug supply of Connecticut.

4.1.4 Drug Co-occurrence Heatmap

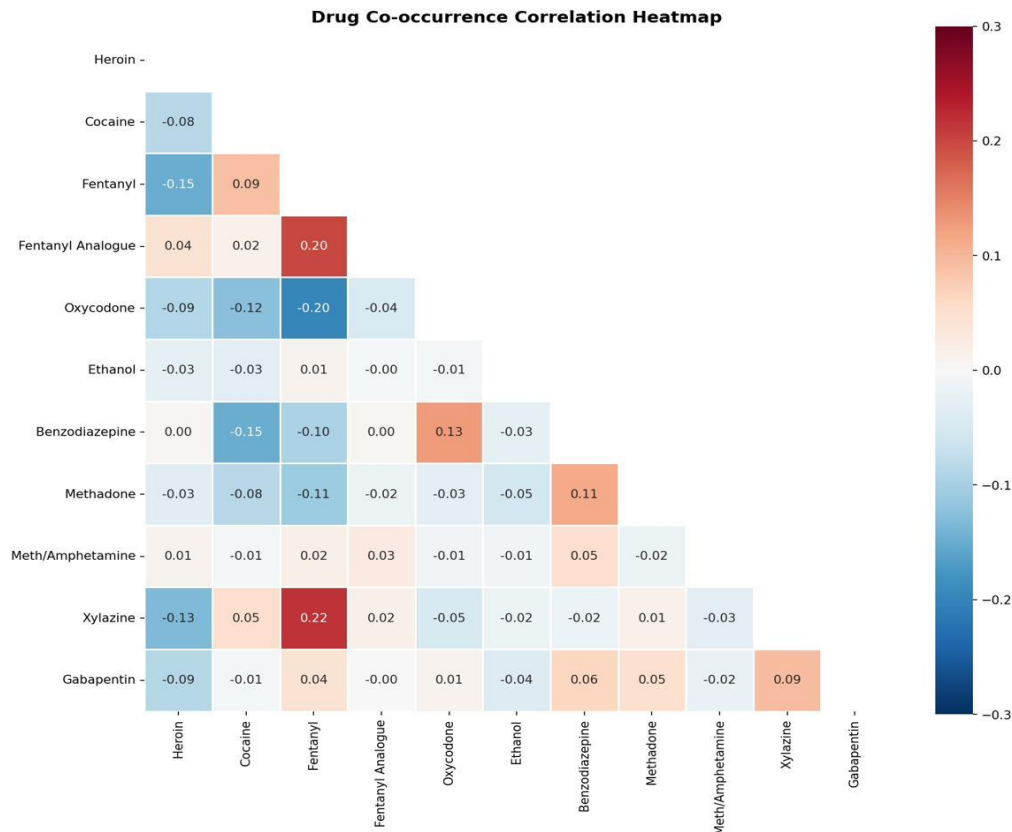


Figure 10: Drug co-occurrence correlation heatmap.

The heatmap shows that fentanyl and xylazine tend to co-occur positively, while heroin and fentanyl show a negative association. These pairwise relationships are useful for understanding how substances appear together in overdose cases, but they should be interpreted as associations rather than proof of causation. Most of the correlations are weak to moderate, so the main value of the heatmap is in highlighting relative co-occurrence patterns rather than very strong dependencies.

The Fentanyl-Xylazine positive association that could be seen in the heatmap is one of the most concerning trends in the current drug supply and it should be interpreted in detail. Xylazine, or tranq, is a veterinary tranquilizer with no human analogue in an overdose, and unopposed by naloxone. To increase potency perceived and the duration of the fentanyl high, suppliers add xylazine to fentanyl to produce a dual overdose effect: despite the successful administration of naloxone to reverse the fentanyl effect, the xylazine effect persists, depressing the central nervous system and leaving respondents in an incorrect state of assurance, that the overdose has been resolved. The positive correlation in his data set validates the idea that cases of fentanyl are

disproportionately likely to also include xylazine, which also has direct clinical implications like emergency responders and colleagues should be trained to keep monitoring patients post-naloxone administration and not to assume that they are in complete recovery. The Heroin-Fentanyl negative correlation is another but equally important discovery: it is the indicator of the long-term-confirmed replacement of heroin by the illegally produced fentanyl in the Connecticut drug market. The negative heroin fentanyl relationship is consistent with the broader shift observed elsewhere in the report, where heroin falls as fentanyl rises over time. The real threat of this replacement is that the users used to the strength of heroin will overdose horrifically, once they are not aware of the presence of fentanyl since the lethal dose level is much less. Reduction programmes in harm reduction should see to it that fentanyl test strips are available in large quantities and that the community messages are clear and explain that what was once perceived to be the local supply of heroin is probably fentanyl.

4.1.5 Geographic Cluster Map

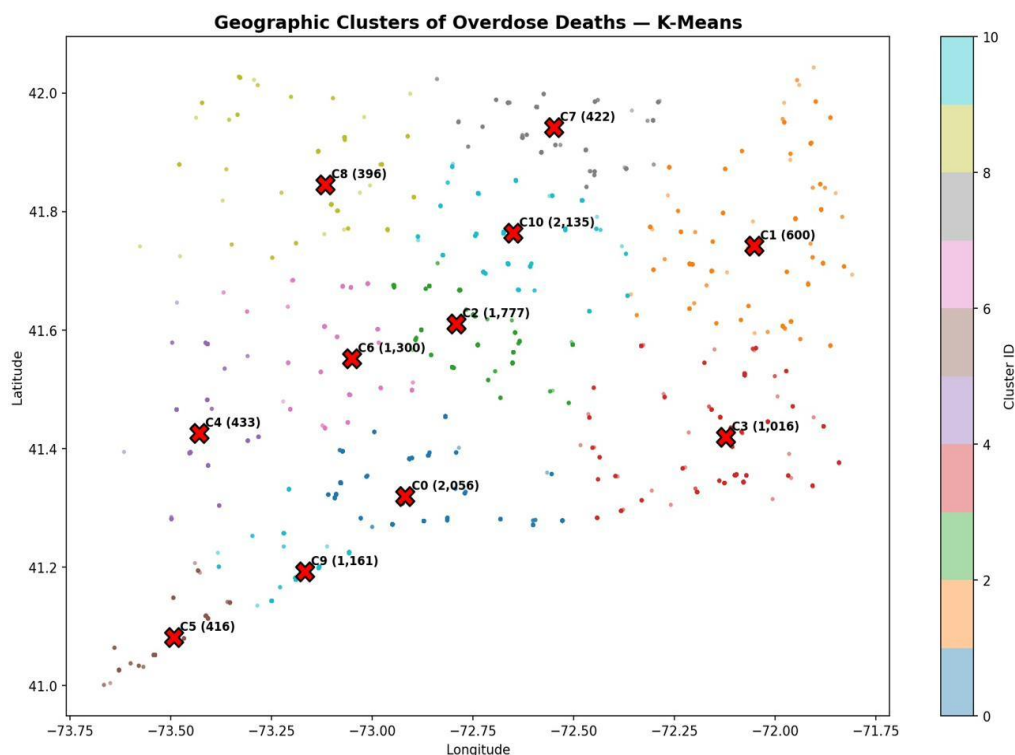


Figure 11: Geographic clusters of overdose deaths with cluster sizes.

In this map, colored points represent overdose cases assigned to K-means clusters, red X markers show the cluster centroids, and the labels indicate the approximate size of each cluster.

The map shows major concentrations of overdose deaths around Hartford and along the New Haven Bridgeport coastal corridor, with cluster labels indicating substantial differences in cluster size. These areas would be reasonable priorities for naloxone distribution and mobile treatment planning because they contain the largest recorded concentrations of cases in the clustering output because the figure is based on clustered case counts, it shows where deaths are concentrated geographically, not where overdose risk is necessarily highest on a per-capita basis.

To translate the cluster map into actual policy recommendations, it will be necessary to interpret the location and the size of the various hotspots identified. The highest density of naloxone dispensing points, including over-the-counter pharmacies, emergency department co-prescription programmes, and syringe service programmes, should be concentrated into the Hartford metro cluster which is associated with the highest single concentration of the deaths. New Haven-Bridgeport coastal corridor is the second priority area: these are neighboring population hubs linked with commuter transit facilities, indicating that mobile outreach units travelling through the Route 1 and I-95 corridor may reach several vulnerable communities on a single deployment plan. The proportional sizes of clusters give a factual basis to the budget allocation, a cluster of Hartford (with about 2,135 deaths) should have a proportionately higher share of the capacity in the treatment facilities than a smaller inland cluster of 396 deaths. This quantitative proportionality shifts the resource allocation of choices between political bartering and anecdote into evidence-based planning.

4.1.6 Anomaly Detection Scatter Plots

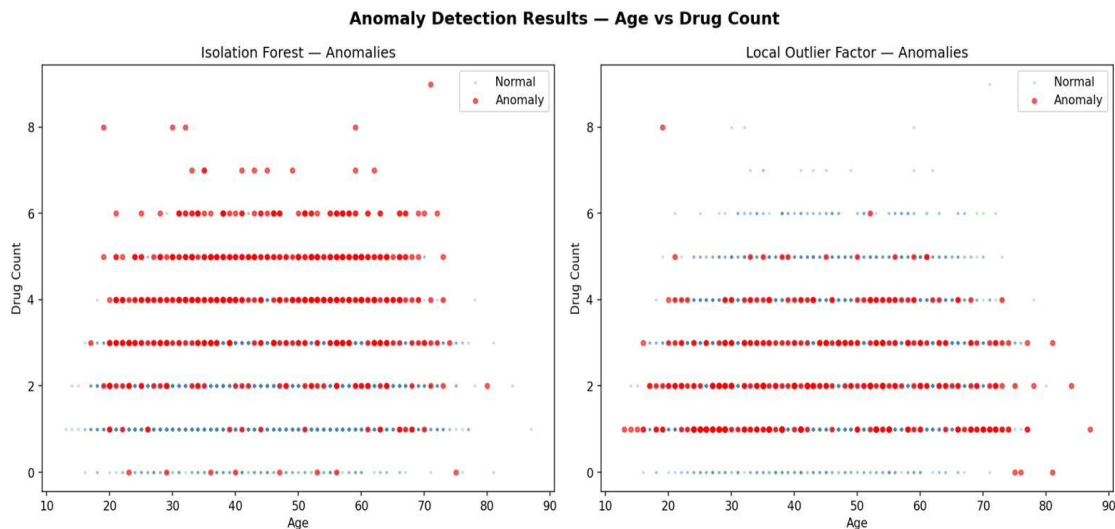


Figure 12: Anomaly scatter plots: Isolation Forest (left) and LOF (right).

The IF anomalies are clumped in the upper part (high Drug_Count) -polysubstance outliers. LOF anomalies vary differently in demographic dimensions. Because the plot is a 2D projection using Age and Drug_Count, it illustrates the anomaly patterns visually but does not represent the full 10-feature space used by the models. The visual disparity attests to the 1.5 percentage point overlap: algorithmic systems pick up radically different types of anomaly.

This graphical deviation is the empirical validation of the structural forecast of Section 2.4: since Isolation Forest assesses global rarity and LOF assesses local density deviation, they ought to identify different populations - and Figure 12 demonstrates precisely that. The IF scatter plot concentrates its flagged cases at the upper end of the Drug_Count axis, which is the location at which the space of global features is the most sparse in the entire 11,968-record dataset: cases with four or more simultaneous substances are globally anomalous at the entire 11,968-record dataset. The LOF scatter plot spreads its anomalies more widely across the population space, indicating cases that are locally unusual relative to their nearest neighbours, even if they are not globally rare in the dataset. The 1.5% overlap is useful because it shows that two methods are detecting mostly different kinds of anomalies, the structural indication that two approaches that aim at the 5% rate of contamination are examining two entirely different phenomena.

The practical implication is that the two populations of anomalies should have different responses of public health. The IF population - with high Drug_Count (mean 4.07) and xylazine rate of 42.4%

- are extreme polysubstance cases, where the primary intervention is harm reduction: fentanyl test strip distribution, xylazine wound care service and polysubstance treatment programmes that targets the multiplicity of the dependencies, and not a single substance. This group cannot be treated with the usual opioid only treatment pathways. The LOF population, older men with non-typical demographic characteristics compared to their local neighbourhood in feature space, might be the people whose situations of overdose do not follow the common pattern in the local community, or whose windows are represented by an unusual demographic context, indicating a new at-risk group. It would be wrong to treat these two populations in the same manner as the data would not be specific. The separation of the two scatter plots in Figure 12 is not a visualisation puzzle - it is the methodology underlying the development of two programs of interventions based on the same set of data.

5. Conclusion

5.1 Summary of Findings

Classification: SVM showed worse results than random forest (F1: 0.78 vs 0.75, AUC: 0.74 vs 0.73). `class_weight='balanced'` was critical for the 74.7% class imbalance. Top predictors: Age (0.33), Drug_Count (0.25), Xylazine (0.13).

Prediction: The prediction of all 4 substances showed that Polynomial regression was better than linear. The most dramatic improvement was on heroin improvement (R^2 : 0.15 - 0.87). The highest is dominated by Fentanyl at +133 cases/year. The xylazine is also a threat that is developing framework of reporting (+32 cases/year since 2019).

Clustering: K-Means found 11 balanced regions; DBSCAN found 12 density-based hotspots. The two are united around the top hotspots of Hartford, New Haven, Bridgeport (ARI=0.54). DBSCAN also found 1,323 solitary rural cases.

Anomaly Detection: 5% were anomalous. IF identifies cases of polysubstance (DrugCount 4.07, Xylazine 42.4%). LOF identifies demographic anomaly (younger males, age 42.0). There is only 1.5% overlap - complementary coverage. The low overlap poses an epistemological question of interest, in that, given that two methods that are targeted at the same contamination rate detect fundamentally different cases, at least one must be detecting patterns that the other is structurally blind to - but neither can be independently validated using ground-truth labels. This renders the complementary deployment decision defensible, but not provable, which can be overcome in the future by labelled anomaly datasets.

These procedures, however, are not uniformly converging, and the separation is also didactic. There is an agreement between K-Means and DBSCAN on where Hartford and the New Haven-Bridgeport region are the main hotspots, but different on the treatment of rural deaths: K-Means puts all 1,323 of them at the closest urban centroid, whereas DBSCAN insists on treating them as noise. It is not the rightness and wrongness of one approach and the other approach- it is the

existence of two algorithms that have different structural assumptions that indicate two different true things about the same deaths. This divergence demonstrates that the rural deaths have a truly ambiguous geographic location: they are near enough to clusters of urban centres to be clustered with them using a distance-minimising algorithm, but are sufficiently sparse in their immediate local neighbourhood to be clustered with using a density-based algorithm. In this respect, the point where K-Means and DBSCAN disagree is more educative than where they agree: it determines exactly what deaths fit between intervention categories neither urban facility-based programmes nor rural cluster-based programmes apply, but individual-targeted mobile outreach does. An interpretation of methodological divergence as an added dimension, as opposed to the lack of agreement, is not the source of weakness of the analysis but rather an added layer of insights.

5.2 Limitations

The restrictions of this study are quite different in the intensity of their impact and the extent of influence on the validity of certain results. These are discussed here in decreasing descending analytical impact.

The first and the most basic weakness is the contamination rate which is user-defined (5%), to both Isolation Forest and Local Outlier Factor. Contamination=.05 specifies that each algorithm is to show 5% of the cases anomalous by default, irrespective of the actual percentages of anomalies in the data. Since there is no labelled ground truth that is provided on the abnormal death of drugs, we will never be able to check whether 5 percent is the right figure, it is the underestimation or even overestimation. This forms a circular form: we demanded 5% of anomalies and we got 5% anomalies. The conclusion that 599 anomalies were identified in each approach is thus due to the setting and not an empirical discovery on its own. This does not mean that the profiling of those anomalies (their DrugCount, age, and xylazine properties) is invalid, however, it implies that the count should not be taken as a valid indication of the actual prevalence of the anomaly in the state of Connecticut of the overdose deaths.

The second greatest limitation is the constraint of regression analysis of 12 data points. Although the use of the method of polynomial regression was specifically due to the fact that ARIMA needs a greater number of observations to be conducted to assure reliable tests of stationarity, there is

also a limitation of the reliability of the estimate of the coefficient of the polynomials by the 12 annual values. The fact that R^2 improvements of Heroin (0.15 - 0.87) are impressive must be viewed with reservations: a degree-2 fitted to 12 points has few degrees of freedom left after estimating the parameters, and the seeming quality of the fit might not be applicable to future years should the trajectory start veering off.

One limitation of moderate severity is that, in nominal features, like Race, the ordinal constraint of LabelEncoder can have caused artefacts of distance in K-Means clustering and SVM classification. This was a calculated trade-off to accommodate the tree-based model as was discussed in Section 1.4.2, but it is a source of possible bias. About 2% of non-geographically-coordinated records were not clustered and this might under-represent rural death in the spatial analysis. The completeness of toxicology is also acknowledged as an issue: there is a possibility that not all substances are identified by medical examiners in an incident, so prevalence rates of substances (especially of more recent adulterants such as xylazine) will be lower than they truly are.

Geographic scopes is the least restrictive limitation: the sample is restricted to Connecticut alone, and results cannot be expected to be applicable to other states with different networks of drug supply, ratios of rural to urban areas, or demographic makeup. This weakness in no way calls the validity of the Connecticut-specific findings into question but limits their immediate generalizability to other jurisdictions without reproduction.

5.3 Suggestions for Future Work

Future directions might use SMOTE to balance the classes, SHAP value to understand the importance of features to be used, LSTM to analyse data over time (using monthly data), multi-state comparisons, socioeconomic characteristics (income, unemployment, insurance status), and spatial autocorrelation to do a more rigorous geographic analysis.

Among such directions, the SHAP (SHapley Additive exPlanations) values are the most high-priority methodological upgrade. The Gini impurity scores of the features that are reported in this

research (0.33 and 0.25 respectively) have a systematic bias towards high-cardinality features: variables with a large number of distinct values (such as continuous Age) will be artificially inflated with an importance score merely because they can provide more candidate split points, whether or not they actually contribute to the prediction. The game-theoretic model of SHAP has its roots in Shapley values of cooperative game theory, and assigns predictive credit to each feature according to its marginal contribution to all possible feature combinations, which prevents the cardinality bias completely. Substituting Gini importance with SHAP values would provide a better attribution of which features actually propel opioid classification, and may indicate that the prevalence of Age is not necessarily an artefact of its continuous scale and not necessarily related to its clinical importance. The second priority would be LSTM temporal modelling, subject to the availability of monthly case data on a monthly basis as opposed to annual data, which would have the minimum sequence length that would guarantee reliable gradient propagation.

6. Work Breakdown Structure

Task	Assigned To	Status
Data Preprocessing (Section 1.4)	Member ____	Complete
Classification – RF + SVM (Section 2.1)	Member ____	Complete
Prediction – Regression (Section 2.2)	Member ____	Complete
Clustering – K-Means + DBSCAN (Section 2.3)	Member ____	Complete
Anomaly Detection – IF + LOF (Section 2.4)	Member ____	Complete
Evaluation & Validation (Section 3)	Member ____	Complete
Data Visualisation (Section 4)	Member ____	Complete
Report Writing & Formatting	Member ____	Complete
Video Presentation	All Members	Complete

Table 6: Work Breakdown Structure.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Centers for Disease Control and Prevention. (2023). Understanding the opioid overdose epidemic. <https://www.cdc.gov/opioids/basics/epidemic.html>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
<https://doi.org/10.1007/BF00994018>
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD-96*, 226–231.
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *Proceedings of the Eighth IEEE International Conference on Data Mining*, 413–422.
<https://doi.org/10.1109/ICDM.2008.17>
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
[https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)